

UNIVERSIDAD DEL BÍO-BÍO

FACULTAD DE CIENCIAS EMPRESARIALES

Departamento de Sistemas de Información

Flujo de clasificación de Audio y Machine-learning Avanzado (F.A.M.A)

AnteProyecto de Título

Dirigida por

Mayo 2026

Resumen

Debe describir su proyecto, resultados y beneficios esperados.
Palabras Clave — xxxx,xxxxx,xxxxx

Agradecimientos

Debe describir su proyecto, resultados y beneficios esperados.

Acrónimos

Índice General

1. Introducción **1**
- 1.1. Presentación del área y su problemática . . . 1
  - 1.1.1. Presentación de la Empresa . . . 1
  - 1.1.2. Presentación de los procesos del ámbito del proyecto en la Empresa . . . 1
  - 1.1.3. Descripción del problema . . . 2
- 1.2. Propuesta de solución . . . 3
- 1.3. Análisis de trabajos relacionados o soluciones existentes . . . 3
- 1.4. Justificación de la propuesta . . . 3
- 1.5. Objetivo general del proyecto . . . 4
- 1.6. Objetivos específicos del proyecto . . . 4
- 1.7. Descripción de las actividades para lograr los objetivos específicos . . . 4

2. Proyecto **6**
- 2.1. Objetivo general del software . . . 6
- 2.2. Objetivos específicos del software . . . 6
- 2.3. Metodología de desarrollo . . . 7
- 2.4. Estándares de documentación . . . 7
- 2.5. Técnicas y notaciones . . . 7
- 2.6. Tecnologías utilizadas: Herramientas, frameworks y lenguajes . . . 7

3. Especificación de Requerimientos - Producto de Software **10**
- 3.1. Límites del proyecto . . . 10
- 3.2. Restricciones técnicas . . . 10
- 3.3. Identificación de usuarios . . . 11
- 3.4. Requerimientos funcionales . . . 12
- 3.5. Requerimientos no funcionales . . . 14
- 3.6. Interfaces externas de entrada . . . 14
- 3.7. Interfaces externas de salida . . . 15

4. Análisis - Casos de Uso **16**
- 4.1. Factibilidad técnica . . . 16
- 4.2. Factibilidad operativa . . . 17

4.3. Factibilidad económica . . . 18
4.3.1. Flujo de caja . . . 18
4.3.2. Cálculo del VAN . . . 19
4.4. Conclusión de factibilidad . . . 20

5. Diseño **21**
5.1. Actores de Casos de Uso . . . 21
5.2. Diagramas y Especificación de Casos de Uso . . . 22
5.2.1. Caso de Uso 1: Iniciar sesión . . . 25
5.2.2. Caso de Uso 2: Gestionar cuenta y perfil . . . 25
5.2.3. Caso de Uso 3: Iniciar sincronización y procesamiento de datos . . . 26
5.2.4. Caso de Uso 4: Seleccionar dataset para entrenamiento . . . 27
5.2.5. Caso de Uso 5: Entrenar modelo acústico . . . 28
5.2.6. Caso de Uso 6: Consultar historial y métricas . . . 29
5.2.7. Caso de Uso 7: Seleccionar modelo activo . . . 29
5.2.8. Caso de Uso 8: Realizar predicción acústica . . . 30
5.2.9. Caso de Uso 9: Retroalimentar predicción errónea . . . 31
5.2.10. Caso de Uso 10: Gestionar usuarios . . . 32
5.2.11. Caso de Uso 11: Gestionar nodos de procesamiento local . . . 33
5.2.12. Caso de Uso 12: Gestionar recursos cloud . . . 33
5.2.13. Caso de Uso 13: Monitorear pipelines y registros . . . 34
5.2.14. Caso de Uso 14: Controlar ejecución de procesos . . . 35
5.3. Matriz de Trazabilidad . . . 35

6. Desarrollo del Trabajo **36**
6.1. Descripción de los servicios web . . . 36
6.1.1. Diagrama del proyecto . . . 36
6.2. Modelo de datos . . . 37
6.2.1. Esquema de base de datos . . . 37
6.2.2. Entidad-Relación . . . 42
6.2.3. Modelo-Relación . . . 45
6.3. Diseño de interfaz y navegación . . . 47
6.3.1. Guías de estilo . . . 47
6.3.2. Logotipo . . . 47
6.3.3. Guía de colores . . . 48
6.3.4. Tipografía . . . 50
6.3.5. Composición de interfaces . . . 51
6.4. Diseño de arquitectura . . . 53
6.4.1. Configuración Inicial del Entorno . . . 53
6.4.2. Arquitectura Implementación Final . . . 54
6.5. Porcentaje de avance por actividad . . . 57
6.6. Programación de actividades futuras . . . 58

7. Conclusiones **60**
7.1. Trabajo Futuro 60

Referencias **61**

A. Definiciones y abreviaciones del Negocio **62**

B. Pruebas de Aceptación **63**

C. Recopilación de Información **64**

D. Diccionario de datos **65**

E. Aspectos de gestión de proyectos **66**
E.1. Carta Gantt con línea base y desviaciones 66
E.2. Riesgos de Alto nivel (Amenazas), Impacto, estrategia 66
E.3. Estimación CU 66
E.4. Resumen Esfuerzo 67
E.5. Retrospectiva Proyecto 67
E.6. Iteraciones en el desarrollo 68

Índice de Figuras

4.1. Fórmula Matemática del Valor Actual Neto (VAN) 19

5.1. Diagrama modulo de sesión de casos de uso 22
5.2. Diagrama modulo de investigador de casos de uso 23
5.3. Diagrama modulo de administrador de casos de uso 24

6.1. Diagrama de servicios web e interconexión operativa del proyecto. 37
6.2. Diagrama Entidad-Relación conceptual del sistema F.A.M.A. 44
6.3. Diagrama del Modelo Relacional de la base de datos. 46
6.4. Diseño tipográfico y composición del logotipo de F.A.M.A. 48
6.5. Vista del Dashboard 51
6.6. Vista de ingesta de datos 52
6.7. Vista de predicción de datos 52
6.8. Vista de entrenamiento del modelo 53
6.9. Diagrama físico de F.A.M.A 55
6.10. Diagrama lógico de F.A.M.A 56

Índice de Tablas

2.1. Resumen del stack tecnológico utilizado en el desarrollo del proyecto F.A.M.A. . 9

3.1. Requerimientos Funcionales del Framework MLOps Híbrido 13
3.2. Requerimientos No Funcionales del Framework 14
3.3. Interfaces Externas de Entrada 15
3.4. Interfaces Externas de Salida 15

4.1. Especificación de Hardware local requerido para el desarrollo y entrenamiento. . 17
4.2. Especificación de Infraestructura Cloud requerida para el proyecto. 17
4.3. Costos de Licencias de Software. 18
4.4. Costo estimado de infraestructura híbrida (GCP Storage). 18
4.5. Cálculo del costo de desarrollo (Inversión Inicial valorizada). 18
4.6. Proyección de Flujo de Caja a 5 años (Cifras en CLP). 19
4.7. Términos de la fórmula de VAN aplicados al proyecto. 19
4.8. Cálculo del flujo descontado anual. 20

5.1. Especificación del Caso de Uso CU_SES_01: Iniciar Sesión 25
5.2. Especificación resumida del Caso de Uso CU_SES_02: Gestionar Cuenta y Perfil 25
5.3. Especificación del Caso de Uso CU_INV_01: Iniciar Sincronización y Procesamiento 26
5.4. Especificación resumida del Caso de Uso CU_INV_02: Seleccionar Dataset para Entrenamiento 27
5.5. Especificación del Caso de Uso CU_INV_03: Entrenar Modelo Acústico 28
5.6. Especificación resumida del Caso de Uso CU_INV_04: Consultar Historial y Métricas 29
5.7. Especificación resumida del Caso de Uso CU_INV_05: Seleccionar Modelo Activo 29
5.8. Especificación del Caso de Uso CU_INV_06: Realizar Predicción Acústica . . . 30
5.9. Especificación del Caso de Uso CU_INV_07: Retroalimentar Predicción Errónea 31
5.10. Especificación del Caso de Uso CU_ADM_01: Gestionar usuarios 32
5.11. Especificación resumida del Caso de Uso CU_ADM_02: Gestionar nodos de procesamiento local 33
5.12. Especificación resumida del Caso de Uso CU_ADM_03: Gestionar recursos cloud 33
5.13. Especificación del Caso de Uso CU_ADM_04: Monitorear pipelines y registros . 34

5.14. Especificación resumida del Caso de Uso CU_ADM_05: Controlar ejecución de procesos . . . 35
5.15. Matriz de trazabilidad: Requerimientos Funcionales - Casos de Uso . . . 35

6.1. Tipos enumerados (ENUMs) . . . 37
6.2. Tabla usuario . . . 38
6.3. Tabla nodo_procesamiento . . . 38
6.4. Tabla conjunto_datos . . . 39
6.5. Tabla audio . . . 39
6.6. Tabla modelo . . . 40
6.7. Tabla metrica_entrenamiento . . . 40
6.8. Tabla prediccion . . . 41
6.9. Tabla retroalimentacion . . . 41
6.10. Tabla registro_pipeline . . . 42
6.11. Modelo Relacional Normalizado de F.A.M.A. . . . 45
6.12. Paleta de colores estructural de la interfaz de usuario. . . . 48
6.13. Uso semántico del color para la comunicación de estados. . . . 49
6.14. Especificaciones tipográficas de la interfaz web de F.A.M.A. . . . 50
6.15. Decisión de infraestructura por capa del sistema. . . . 54
6.16. Estructura modular del Backend. . . . 57
6.17. Porcentaje de avance por actividad según la planificación inicial. . . . 58

Capítulo 1

Introducción

El presente capítulo introduce el contexto, la problemática y los fundamentos que motivan la realización de este proyecto de titulación. Se detalla la situación actual del entorno de investigación, la propuesta de valor basada en una arquitectura híbrida, los objetivos a alcanzar y las actividades planificadas para construir el software de forma metódica. Finalmente, se describe la estructura general del documento.

1.1. Presentación del área y su problemática

1.1.1. Presentación de la Empresa

El presente proyecto de desarrollo de software nace como una iniciativa propia de investigación y desarrollo, gestada en el entorno formativo de la carrera de Ingeniería Civil en Informática de la Universidad del Bío-Bío (UBB). Al no contar con un patrocinio corporativo externo, el proyecto opera bajo un modelo de autogestión tecnológica, donde el equipo desarrollador asume el rol de la organización para efectos de la formulación de este documento. El objetivo principal de esta iniciativa es proveer una solución arquitectónica avanzada e independiente orientada a investigadores, académicos y estudiantes que requieran orquestar el procesamiento masivo de señales acústicas y el entrenamiento de modelos de Inteligencia Artificial (IA), eliminando la dependencia de infraestructuras corporativas o servicios en la nube de alto costo.

1.1.2. Presentación de los procesos del ámbito del proyecto en la Empresa

Dentro del alcance de esta iniciativa, específicamente en el ámbito de la clasificación de audio mediante Inteligencia Artificial, se ha identificado que los procesos actuales de investigación a nivel académico y de desarrollo temprano se ejecutan de manera predominantemente manual y secuencial, careciendo de automatización y centralización. El flujo de trabajo operativo tradicional que enfrentan los desarrolladores se sustenta en el uso de herramientas de consumo propio y *scripts* aislados, operando sin un repositorio centralizado de datos ni un canal de ejecución trazable.

A continuación, se describen los pasos principales del procedimiento actual que esta iniciativa busca resolver:

(a) **Búsqueda y descarga estática:** El investigador acude a diversas fuentes en internet para buscar y descargar de forma estática bancos de audios acústicos crudos hacia su equipo local.
(b) **Preprocesamiento aislado:** Utilizando *scripts* y herramientas de desarrollo propias, el usuario limpia, formatea y procesa los archivos acústicos de forma no estandarizada.
(c) **Entrenamiento local y cuellos de botella:** El investigador entrena modelos matemáticos complejos de *Deep Learning* en su hardware personal, sufriendo constantes cuellos de botella al saturar la memoria y la capacidad de procesamiento.
(d) **Balanceo manual e ineficiente:** Esta limitación tecnológica fuerza al investigador a reducir drásticamente los conjuntos de datos estáticos para lograr su balanceo, lo que provoca una pérdida masiva de audios útiles para el aprendizaje de la IA.
(e) **Evaluación e inseguridad productiva:** Una vez finalizado el entrenamiento, el usuario somete el modelo a pruebas algorítmicas, obteniendo a menudo porcentajes de precisión insuficientes para entornos productivos debido al sobreajuste (overfitting) derivado del uso de datos limitados.
(f) **Falta de trazabilidad y replicabilidad:** La gestión ineficiente del ciclo de vida del *Machine Learning* recae en ejecuciones manuales, lo que imposibilita la replicabilidad exacta de los experimentos y la trazabilidad histórica de los modelos obtenidos.

1.1.3. Descripción del problema

En el ámbito del aprendizaje automático acústico, el desarrollo de modelos enfrenta deficiencias críticas derivadas de la fragmentación del flujo de trabajo y la dependencia de infraestructuras exclusivamente locales o herramientas genéricas desconectadas.

Actualmente, el entrenamiento de modelos complejos de *Deep Learning* genera severos cuellos de botella al saturar rápidamente los recursos de memoria y procesamiento del *hardware* personal. Esta limitación tecnológica fuerza a los investigadores a reducir drásticamente los conjuntos de datos, provocando pérdida de información y derivando en modelos sobreajustados (*overfitting*). A esta barrera técnica se suma la gestión ineficiente del ciclo de vida del dato, recayendo en *scripts* manuales susceptibles a errores humanos.

Si bien la solución evidente en la industria es migrar el procesamiento a la nube (soluciones *Cloud-Native*), en el entorno académico esto presenta dos problemas. Primero, plataformas como Google Vertex AI exigen conocimientos avanzados en arquitectura *cloud*, dificultando su adopción. Segundo, el costo por hora de arrendar instancias de cómputo con aceleración gráfica (GPUs) resulta económicamente prohibitivo e inviable para los presupuestos estándar de investigación.

Por consiguiente, la problemática radica en la carencia de una infraestructura unificada que actúe como puente entre ambos mundos: que permita aprovechar el almacenamiento económico y estandarizado de la nube, sin perder la rentabilidad de utilizar el cómputo local disponible para el entrenamiento de los algoritmos.

1.2. Propuesta de solución

Para dar solución a la problemática identificada, se propone el diseño y desarrollo de F.A.M.A., un *framework* MLOps de **Arquitectura Híbrida**, enfocado en la clasificación de señales acústicas. La implementación de esta tecnología resuelve las deficiencias actuales al transicionar de un enfoque manual hacia un sistema orquestador automatizado.

El sistema utilizará los servicios de Google Cloud Platform (GCP) para actuar como la fuente de verdad centralizada, almacenando masivamente los *datasets* a un costo marginal. En paralelo, se desarrollará un *software* orquestador en Python que descargará los lotes de manera segura, ejecutará el preprocesamiento matemático y el entrenamiento de las redes neuronales utilizando exclusivamente el *hardware* local del usuario, para finalmente mostrar las predicciones en un entorno visual local interactivo.

Adicionalmente, el *framework* incorporará un módulo de telemetría y ciclo de retroalimentación (*feedback loop*) orientado al aprendizaje continuo. Este mecanismo permitirá que, ante una clasificación errónea, el investigador registre la corrección directamente desde la interfaz web. El sistema capturará de forma automática el archivo de audio junto con su nueva etiqueta corregida y los respaldará en el repositorio centralizado en la nube, estructurando así un flujo de datos dinámico optimizado para futuros ciclos de re-entrenamiento automatizado sin destruir el conjunto de datos original.

1.3. Análisis de trabajos relacionados o soluciones existentes

A nivel institucional, los desarrollos previos para la clasificación acústica se han basado en enfoques locales. Al operar exclusivamente con *scripts* en la máquina del investigador, estos trabajos sufren la saturación de la memoria de video (VRAM) y se ven obligados a limitar la cantidad de datos, impidiendo su escalabilidad y colaboración.

Por otro lado, a nivel comercial, destacan las soluciones genéricas como AWS SageMaker. Estas plataformas resuelven el desorden de datos, pero transfieren un costo monetario sumamente elevado por cada minuto de procesamiento matemático en sus servidores. En este contexto, la arquitectura híbrida propuesta se posiciona como el punto de equilibrio óptosimo, resolviendo la fragmentación con almacenamiento *cloud* económico, y evadiendo los costos de servidores delegando el esfuerzo pesado al entorno local.

1.4. Justificación de la propuesta

Técnicamente, la implementación de este *framework* MLOps híbrido resulta imperativa para estandarizar el ciclo de vida del audio, garantizando que los experimentos sean ordenados, trazables y fácilmente reproducibles.

En el ámbito económico, la propuesta ofrece una gestión significativamente más eficiente de los recursos institucionales. Al evitar el modelo 100 % nube, el proyecto elimina los altísimos costos asociados al arrendamiento de GPUs comerciales. El modelo extrae un beneficio máximo

pagando únicamente por el almacenamiento estático, mientras amortiza y explota al máximo las capacidades de las tarjetas gráficas locales con las que ya cuenta la institución o los estudiantes.

Finalmente, desde un enfoque metodológico y social, la herramienta impulsa una democratización tecnológica. Al encapsular la complejidad de la conexión nube-local y la matemática de los audios en una interfaz visual, cualquier investigador podrá acelerar sus ciclos de experimentación, facilitando futuros desarrollos con impacto real, como sistemas para diagnóstico médico respiratorio o mantenimiento industrial predictivo.

1.5. Objetivo general del proyecto

Implementar un *pipeline* MLOps de arquitectura híbrida que automatice la gestión y clasificación de señales acústicas, combinando el almacenamiento centralizado en Google Cloud Platform con la orquestación del entrenamiento en infraestructura local.

1.6. Objetivos específicos del proyecto

(a) Analizar el estado del arte respecto a arquitecturas MLOps híbridas, plataformas de almacenamiento en la nube y técnicas de procesamiento digital de señales acústicas.
(b) Levantar los requerimientos funcionales y de infraestructura necesarios para la orquestación segura entre el repositorio alojado en la nube y los entornos de ejecución locales.
(c) Diseñar la arquitectura lógica y el modelo de flujo de datos del sistema, definiendo los puntos de integración, las responsabilidades de cada entorno y la estructura del ciclo de retroalimentación.
(d) Construir los módulos de *software* encargados de la sincronización de archivos, preprocesamiento de características, la interfaz gráfica de validación local y el componente de captura de telemetría de errores para la mejora continua.
(e) Evaluar el rendimiento, la viabilidad y la efectividad del ciclo de re-entrenamiento del *pipeline* híbrido mediante pruebas de concepto, analizando la evolución de la precisión del modelo y el ahorro de costos frente a alternativas puramente en la nube.

1.7. Descripción de las actividades para lograr los objetivos específicos

Para asegurar el cumplimiento de las metas trazadas, se definen las siguientes actividades organizadas por objetivo específico:

- **Actividades OE1.**
  - Revisar literatura técnica sobre metodologías MLOps híbridas y flujos de trabajo de *Machine Learning*.
  - Investigar técnicas de procesamiento digital de señales acústicas, específicamente extracción de espectrogramas y coeficientes MFCC.

  - Analizar y comparar costos entre infraestructuras puramente *Cloud-Native* (ej. Vertex AI) e infraestructuras locales apoyadas en almacenamiento en la nube.
- **Actividades OE2.**
  - Recopilar necesidades mediante entrevistas y reuniones de avance con el profesor guía e investigadores del área.
  - Redactar requerimientos funcionales y no funcionales basados en la adaptación del estándar IEEE 830.
  - Definir restricciones operativas y de infraestructura, detallando el uso de GPUs locales, autenticación IAM en Google Cloud y límites de almacenamiento.
  - Identificar a los actores del sistema (Investigador y Administrador) y documentar formalmente sus interacciones iniciales.
- **Actividades OE3.**
  - Especificar la separación lógica de las capas del sistema (*backend* orquestador, almacenamiento *cloud* e interfaz gráfica local).
  - Modelar el sistema mediante diagramas de Lenguaje Unificado de Modelado (UML), abarcando Casos de Uso, Actividad y Componentes.
  - Diseñar el modelo entidad-relación para la base de datos (PostgreSQL) encargada de persistir perfiles, metadatos y trazabilidad de los modelos entrenados.
- **Actividades OE4.**
  - Desarrollar los *scripts* de Python para la autenticación e ingesta masiva segura contra la API de Google Cloud Storage.
  - Programar el módulo de preprocesamiento utilizando la librería Librosa para la extracción automatizada de matrices matemáticas.
  - Implementar el *pipeline* de entrenamiento integrando *frameworks* tensoriales (PyTorch/TensorFlow) que exploten la aceleración gráfica del entorno local.
  - Construir la interfaz interactiva utilizando React/Next.js para desplegar vistas de validación visual y carga de audios de prueba.
- **Actividades OE5.**
  - Ejecutar pruebas de concepto procesando un *dataset* acústico real a través de todo el ciclo de vida del *framework* (ingesta, entrenamiento y predicción).
  - Medir y documentar métricas de precisión de la red neuronal, latencia de sincronización y uso de memoria en el hardware local.
  - Evaluar la viabilidad económica calculando el Flujo de Caja y el Valor Actual Neto (VAN) derivado del ahorro en servidores *cloud*.

Capítulo 2

Proyecto

El presente capítulo tiene por objetivo detallar la concepción y estructuración del software propuesto. Se describen los objetivos técnicos del sistema, la metodología de trabajo seleccionada, los estándares de documentación adoptados y, finalmente, se realiza un análisis profundo del stack tecnológico, frameworks y herramientas que sustentan la arquitectura híbrida.

2.1. Objetivo general del software

Desarrollar un software orquestador híbrido que automatice el flujo de trabajo de Machine Learning (MLOps), gestionando la sincronización de audios desde la nube y controlando el preprocesamiento, entrenamiento y visualización de modelos predictivos en el entorno local.

2.2. Objetivos específicos del software

(a) Construir un módulo de integración seguro que permita la lectura, descarga y carga bidireccional de datos con Google Cloud Storage, asegurando tanto el almacenamiento de los modelos como el respaldo de los registros de error etiquetados.
(b) Programar rutinas de procesamiento digital para la extracción automatizada de características matemáticas (espectrogramas y coeficientes MFCC) a partir de los audios crudos.
(c) Desarrollar el pipeline de entrenamiento local que alimente las redes neuronales con los datos procesados, exporte las métricas de rendimiento resultantes y permita la reincorporación de los lotes de datos corregidos.
(d) Implementar una interfaz de validación y control (Dashboard) que permita visualizar la ingesta del dato, la predicción en tiempo real y la captura de telemetría de errores mediante un mecanismo de retroalimentación activa para el aprendizaje continuo.

2.3. Metodología de desarrollo

La metodología de desarrollo a utilizar en el proyecto es la Metodología Ágil, basada en el marco de trabajo Scrum, adaptada para un equipo de desarrollo pequeño.

Esta metodología fue seleccionada ya que este proyecto requiere la integración continua de componentes independientes (servicios en la nube, scripts de procesamiento de audio, redes neuronales y una interfaz visual). Trabajar bajo iteraciones cortas (Sprints) permite al equipo validar la conexión con las API de Google Cloud de manera temprana, desarrollar el software de forma modular y pivotar rápidamente ante cualquier problema de compatibilidad con las librerías matemáticas.

2.4. Estándares de documentación

La documentación del sistema se guiará por los estándares de la industria de la ingeniería de software:

- Adaptación basada en IEEE Software Test Documentation Std 829-1998 para la planificación y registro de las pruebas del modelo de Machine Learning.
- Adaptación basada en IEEE Software Requirements Specifications Std 830-1998 para el levantamiento y estructuración formal de requerimientos.

2.5. Técnicas y notaciones

Para modelar la arquitectura y el comportamiento del sistema, se utilizará el Lenguaje Unificado de Modelado (UML). Específicamente, se emplearán Diagramas de Casos de Uso para detallar las interacciones de los investigadores con la plataforma, Diagramas de Actividad para ilustrar el flujo de sincronización de datos con la nube, y Diagramas de Componentes para representar la separación de responsabilidades entre el hardware local y Google Cloud Platform.

2.6. Tecnologías utilizadas: Herramientas, frameworks y lenguajes

La selección del *stack* tecnológico se sustenta en criterios de arquitectura de software enfocados en la alta disponibilidad de datos, la eficiencia en el procesamiento matemático pesado y el diseño de interfaces modernas. Dado el enfoque de Arquitectura Híbrida del proyecto, el sistema integra tecnologías de almacenamiento en la nube con ecosistemas de ejecución local, apoyándose en *frameworks* robustos tanto para el *backend* algorítmico como para la presentación de resultados.

Python

Python es el lenguaje de programación principal elegido para el desarrollo de la lógica de negocio, orquestación MLOps y *Machine Learning*. Su elección se fundamenta en su posición

como el estándar *de facto* en la industria de la ciencia de datos, ofreciendo un vasto ecosistema de librerías optimizadas para el cálculo tensorial y la integración sencilla con las API de Google Cloud, reduciendo los tiempos de desarrollo en la construcción de *scripts* asíncronos.

Librerías de Machine Learning (PyTorch, TensorFlow y Librosa)

Para el núcleo analítico del *framework*, se utilizan herramientas especializadas en inteligencia artificial y señales acústicas. **Librosa** se emplea para el procesamiento digital de la onda de audio, extrayendo los espectrogramas y coeficientes MFCC. Por su parte, **PyTorch** o **TensorFlow** actúan como los motores de *Deep Learning*, proporcionando las estructuras de datos (tensores) y la optimización matemática necesaria para entrenar las redes neuronales aprovechando la aceleración gráfica (GPU) del entorno local.

Google Cloud Storage y Cloud SQL (PostgreSQL)

Para la persistencia de datos y metadatos se adopta un enfoque en la nube de Google. **Cloud Storage** se utiliza como repositorio de objetos inmutables para alojar masivamente los *datasets* de audios y los pesos de los modelos entrenados. Complementariamente, **PostgreSQL** (mediante Cloud SQL) provee un motor relacional sólido para mantener la integridad referencial y el seguimiento histórico de los modelos creados, usuarios registrados y los cruces lógicos entre etiquetas y audios, garantizando el modelo transaccional ACID.

React y Next.js (Frontend)

Para la interfaz de validación y el *dashboard* interactivo, se utilizará **React** apoyado por el entorno **Next.js**. React facilita la construcción de interfaces mediante componentes reutilizables y un Virtual DOM que asegura actualizaciones en tiempo real sin recargar la página. Esto es ideal para visualizar las métricas de entrenamiento, la carga de *datasets* y las predicciones acústicas en vivo de manera fluida y responsiva.

Antigravity y Google Gemini

Para el entorno de construcción, se emplea **Antigravity** como el editor de código principal para el desarrollo del ecosistema completo. Este entorno se complementa estratégicamente con **Google Gemini**, un potente asistente de Inteligencia Artificial generativa. Gemini interviene activamente en el ciclo de desarrollo agilizando la escritura de *scripts* complejos, depurando errores de compatibilidad en librerías matemáticas y optimizando las consultas a la base de datos relacional.

Git y GitHub

Para el control de versiones se utiliza Git como sistema distribuido y GitHub como repositorio remoto. Esto permite a los ingenieros del equipo colaborar de manera estructurada, conservar

un historial completo de las iteraciones del modelo matemático y revisar el código mediante *pull requests*.

Resumen del stack tecnológico

Tecnología	**Categoría**	**Versión**	**Rol en el sistema**
Python	Lenguaje de programación	v3.10 o superior	Lógica backend, orquestación MLOps y *Machine Learning*.
PostgreSQL	Base de datos relacional	v14 o superior	Persistencia e integridad referencial de metadatos de modelos y usuarios.
Google Cloud Storage	Almacenamiento de objetos (Nube)	Standard	Repositorio principal de datasets de audios y binarios de redes neuronales.
PyTorch / TensorFlow	Framework de Deep Learning	Versión estable	Construcción, entrenamiento y validación matemática de la red neuronal.
Librosa	Librería de procesamiento	v0.10 o superior	Transformación de ondas de audio a espectrogramas y MFCC.
React / Next.js	Framework y librería Frontend	v18 / v14 o superior	Construcción de la interfaz gráfica y Dashboard de predicciones.
Git / GitHub	Control de versiones	Git v2 o superior	Versionado distribuido y colaboración en el código fuente.
Antigravity	Editor de Código / IDE	Versión estable	Entorno principal para el desarrollo del código fuente.
Google Gemini	Asistente de IA	Pro / Advanced	Soporte en el desarrollo de código y depuración de algoritmos.

Tabla 2.1: Resumen del stack tecnológico utilizado en el desarrollo del proyecto F.A.M.A.

Capítulo 3

Especificación de Requerimientos - Producto de Software

El presente capítulo define los límites, restricciones y requerimientos funcionales y no funcionales que el framework MLOps híbrido debe cumplir para satisfacer las necesidades operativas de la institución. Se detallan además los perfiles de usuario y las interfaces de entrada y salida de datos del sistema.

3.1. Límites del proyecto

Para acotar el alcance del proyecto y garantizar su viabilidad técnica en los plazos establecidos, se definen los siguientes límites:

- El sistema **no** realizará el entrenamiento de los modelos de Machine Learning en los servidores de Google Cloud Platform; la nube se utilizará exclusivamente para el almacenamiento de datos.
- El software **no** etiquetará automáticamente los audios crudos; los datasets subidos a la nube deben venir previamente clasificados o etiquetados por el usuario.
- El framework se limitará al procesamiento de archivos de audio (formatos como .wav); **no** procesará datos de video ni texto.

3.2. Restricciones técnicas

La arquitectura híbrida cuenta con las siguientes restricciones obligatorias para su funcionamiento:

- **Hardware Local:** El entorno de ejecución local debe contar con una unidad de procesamiento gráfico (GPU) dedicada, preferentemente con soporte CUDA (NVIDIA), para garantizar tiempos de entrenamiento viables.
- **Credenciales Cloud:** El sistema requiere obligatoriamente un archivo de credenciales de cuenta de servicio (JSON) de Google Cloud Identity and Access Management (IAM) con

permisos de lectura/escritura sobre los buckets de Cloud Storage.
- **Conectividad:** Se requiere conexión a internet de banda ancha activa durante las fases de sincronización de datos (ingesta) y subida de métricas.

3.3. Identificación de usuarios

En el contexto del framework MLOps híbrido F.A.M.A., se identifican dos perfiles principales de usuarios que interactuarán directa o indirectamente con la plataforma, cada uno con distintos niveles de permisos y conocimientos técnicos:

- **Investigador (Usuario Estándar):** Corresponde a estudiantes, analistas o desarrolladores industriales enfocados en el dominio acústico. Su principal objetivo es utilizar la plataforma para cargar audios masivos, etiquetar datasets, entrenar o validar modelos predictivos y retroalimentar al sistema validando predicciones en tiempo real. No requieren conocimientos avanzados en infraestructura de servidores o redes, ya que el sistema abstrae dicha complejidad técnica a través de su interfaz.
- **Administrador de Sistemas (Superusuario):** Corresponde al personal técnico encargado del soporte operativo y mantenimiento del ecosistema. Sus responsabilidades incluyen la gestión de accesos, la configuración de los nodos de procesamiento local y el monitoreo de los recursos en Google Cloud Platform. Requieren un nivel técnico avanzado en despliegue de software y administración de arquitecturas Cloud.

3.4. Requerimientos funcionales

La lista de los requerimientos funcionales específicos se presenta en la Tabla 3.1. Cada requerimiento ha sido redactado contestando explícitamente a las preguntas: *Quién, Qué, Qué restricciones existen, Cuándo y Qué pasa después*. Se han resaltado en mayúsculas las palabras clave para facilitar su seguimiento y evaluación.

ID	El sistema debe permitir que:
RF_01	El ORQUESTADOR (Backend) pueda AUTENTICARSE con Google Cloud Storage. El sistema NO REQUIERE interacción manual cada vez, pero debe poseer un archivo JSON válido. Se autentica CUANDO inicia el pipeline. Una autenticación EXITOSA genera un token de sesión seguro, permitiendo el acceso; si falla, el proceso se detiene con error.
RF_02	El ORQUESTADOR pueda DESCARGAR masivamente los datasets de audio desde la nube al entorno local. SÓLO SI existen archivos nuevos o actualizados en el bucket. Se ejecuta CUANDO el investigador inicia la fase de ingesta. La descarga EXITOSA almacena los archivos en un directorio temporal local y genera un registro (log) de sincronización.
RF_03	El ORQUESTADOR pueda EXTRAER características matemáticas (espectrogramas y MFCC) de los audios. SÓLO SI los archivos descargados poseen un formato de audio válido (ej. .wav). Se realiza CUANDO finaliza la descarga. El procesamiento EXITOSO convierte los audios en matrices de datos (tensores) listos para la inteligencia artificial.
RF_04	El INVESTIGADOR pueda ENTRENAR la red neuronal localmente. SÓLO SI los tensores de audio fueron generados correctamente y existe memoria GPU disponible. Se ejecuta CUANDO se invoca el script de modelado. El entrenamiento EXITOSO genera un archivo binario del modelo (.h5 o .pt) y un reporte de métricas de precisión.
RF_05	El INVESTIGADOR pueda VISUALIZAR predicciones acústicas en tiempo real mediante un Dashboard. SÓLO SI se ha cargado previamente un modelo entrenado en el sistema. Se realiza CUANDO el usuario carga un audio de prueba en la interfaz web. El análisis EXITOSO despliega en pantalla la clase detectada (ej. tipo de sonido) y el nivel de confianza de la IA.
RF_06	El INVESTIGADOR pueda RETROALIMENTAR al sistema sobre la exactitud de una predicción. SÓLO SI se acaba de completar el análisis de un audio de prueba. Se ejecuta CUANDO el usuario confirma o corrige la clase detectada en el Dashboard. El proceso EXITOSO almacena el audio validado en Google Cloud Storage y registra su nueva etiqueta en PostgreSQL, dejándolo disponible para futuros reentrenamientos.

Tabla 3.1: Requerimientos Funcionales del Framework MLOps Híbrido

3.5. Requerimientos no funcionales

Los requerimientos que definen los atributos de calidad, rendimiento y seguridad del sistema se detallan en la Tabla 3.2.

ID	Descripción del Requerimiento No Funcional
RNF_01	**Rendimiento de Procesamiento:** El sistema debe ser capaz de transformar un archivo de audio crudo (10 segundos de duración) a su respectivo espectrograma en un tiempo no mayor a 2 segundos utilizando procesamiento por lotes (batch).
RNF_02	**Escalabilidad de Almacenamiento:** El módulo de ingesta debe soportar la descarga y gestión local de datasets de hasta 50 GB sin producir desbordamientos de memoria RAM (Out of Memory).
RNF_03	**Seguridad de Credenciales:** Las llaves de acceso a Google Cloud Platform nunca deben estar hardcodeadas en el código fuente; el sistema debe leerlas a través de variables de entorno (.env) locales para evitar fugas de información en los repositorios de GitHub.
RNF_04	**Usabilidad de la Interfaz:** El Dashboard de validación (desarrollado con React/Next.js) debe desplegar los resultados de la predicción en pantalla en menos de 3 segundos posteriores a la carga del audio de prueba.

Tabla 3.2: Requerimientos No Funcionales del Framework

3.6. Interfaces externas de entrada

Las interfaces externas de entrada especifican los conjuntos de datos que el sistema recibirá para poder operar. Independientemente del método de ingreso (lectura de APIs o carga manual en el Dashboard), en la Tabla 3.3 se detallan los datos críticos de entrada.

Identificador	Nombre del ítem	Detalle de Datos contenidos en el ítem
IE_01	Credenciales IAM (GCP)	PROJECT_ID, PRIVATE_KEY, CLIENT_EMAIL, CLIENT_ID
IE_02	Dataset de Audio crudo	FILE_NAME, AUDIO_WAV_DATA, SAMPLE_RATE, LABEL_CLASS
IE_03	Hiperparámetros del Modelo	LEARNING_RATE, BATCH_SIZE, NUM_EPOCHS
IE_04	Etiqueta de Retroalimentación (Feedback)	AUDIO_FILE_ID, CORRECTED_LABEL_CLASS, IS_CORRECT_BOOLEAN

Tabla 3.3: Interfaces Externas de Entrada

3.7. Interfaces externas de salida

Las interfaces de salida corresponden a la información y artefactos que el sistema entrega hacia el usuario (investigador) o hacia el entorno de almacenamiento, indicando el medio por el cual se exponen.

Identificador	Nombre del ítem	Detalle de Datos contenidos en el ítem	Medio de Salida
IS_01	Modelo Acústico Entrenado	MODEL_WEIGHTS, NEURAL_ARCHITECTURE_DEF	Archivo Binario (.pt / .h5) guardado en disco.
IS_02	Métricas de Entrenamiento	ACCURACY, LOSS, VALIDATION_SCORE, EPOCH_TIME	Gráficos en Consola / Dashboard.
IS_03	Predicción Acústica	PREDICTED_CLASS, CONFIDENCE_PERCENTAGE	Interfaz Gráfica (Dashboard Web).
IS_04	Logs de Sincronización	TIMESTAMP, FILES_DOWNLOADED, CLOUD_BYTES_READ	Archivo de texto (.log).

Tabla 3.4: Interfaces Externas de Salida

Capítulo 4

Análisis - Casos de Uso

El presente capítulo tiene por objetivo evaluar la viabilidad del proyecto desde tres dimensiones fundamentales: técnica, operativa y económica. Este análisis permite asegurar que la institución cuenta con los recursos necesarios para construir, implementar y mantener la arquitectura MLOps híbrida propuesta, garantizando además que la inversión de tiempo y esfuerzo generará un valor positivo comprobable.

4.1. Factibilidad técnica

La factibilidad técnica del proyecto es altamente favorable debido a los siguientes factores:

- **Recurso Humano:** El equipo está conformado por dos estudiantes de último año de Ingeniería Civil en Informática con formación en analítica avanzada y desarrollo backend, poseyendo las competencias necesarias para programar en Python e integrar servicios Cloud.
- **Software:** Todas las librerías de Machine Learning (TensorFlow, PyTorch, Librosa) y los entornos de desarrollo (Antigravity) son de código abierto (*Open Source*) o de libre acceso.
- **Hardware e Infraestructura:** Se cuenta con estaciones de trabajo locales equipadas con tarjetas gráficas dedicadas para suplir la carga de entrenamiento matemático, y la infraestructura de Google Cloud está disponible bajo demanda.

REQUERIMIENTOS	**DESCRIPCIÓN**
Sistema operativo	Windows 10/11 o distribución Linux (Ubuntu 20.04+).
Procesador	Intel Core i5 / AMD Ryzen 5 o superior.
Memoria RAM	16 GB (Recomendado para procesamiento de audio).
Espacio de almacenamiento	50 GB de espacio libre (SSD preferentemente).
Aceleración Gráfica	Tarjeta de Video Dedicada (NVIDIA con soporte CUDA).
Conexión a internet	Requerida (Banda ancha para sincronización de datasets).

Tabla 4.1: Especificación de Hardware local requerido para el desarrollo y entrenamiento.

REQUERIMIENTOS	**DESCRIPCIÓN**
Proveedor Cloud	Google Cloud Platform (GCP).
Servicio de Almacenamiento	Cloud Storage (Buckets Standard/Coldline).
Gestión de Accesos	Google IAM (Identity and Access Management).

Tabla 4.2: Especificación de Infraestructura Cloud requerida para el proyecto.

Dado que todo el hardware local ya forma parte del equipamiento de los estudiantes y la infraestructura de nube es un servicio público de fácil aprovisionamiento, se concluye que el proyecto es técnicamente factible.

4.2. Factibilidad operativa

El proyecto es operativamente factible dado que resuelve una necesidad crítica y real dentro del entorno de investigación académica, adaptándose de manera orgánica a las dinámicas de trabajo de los usuarios. Actualmente, los investigadores reconocen que la gestión manual de los audios, la fragmentación de los experimentos y los bloqueos constantes del hardware son inconvenientes diarios que retrasan significativamente sus avances científicos. La solución propuesta automatiza estas tareas complejas sin obligar al personal a adquirir conocimientos avanzados en infraestructuras de servidores o arquitecturas de nube complejas, asegurando una curva de aprendizaje mínima.

A esto se suma el impacto positivo de la interfaz de retroalimentación activa basada en telemetría de errores. Desde una perspectiva analítica y operativa, este componente eleva la

viabilidad del framework al otorgar a los investigadores el control directo sobre la calidad del modelo de forma simplificada. En lugar de requerir procesos externos de auditoría técnica o flujos complejos de reetiquetado manual desordenado, los usuarios pueden corregir cualquier desviación o clasificación errónea de la Inteligencia Artificial directamente desde el Dashboard interactivo.

El hecho de que el sistema capture de manera transparente estas correcciones y las envíe al almacenamiento centralizado en Google Cloud Platform asegura que el entorno madure y se optimice continuamente con el mismo uso cotidiano. Esto reduce drásticamente el esfuerzo operativo a largo plazo, garantizando una alta tasa de adopción, la sostenibilidad del software en el tiempo y una excelente disposición para su despliegue definitivo en el Departamento.

4.3. Factibilidad económica

Para determinar la viabilidad económica, se desglosan los costos asociados a licencias, infraestructura Cloud y la valorización de las horas de desarrollo.

Software	**Licencia**	**Costo Licencia**
Python	Python Software Foundation License	$0 CLP
Antigravity / React	MIT License	$0 CLP
TensorFlow / Librosa	Apache 2.0 / ISC	$0 CLP

Tabla 4.3: Costos de Licencias de Software.

Item	**$ mensual aprox. en CLP**	**$ anual aprox. en CLP**
Google Cloud Storage (Almacenamiento)	$4.500 CLP	$54.000 CLP
Transferencia de red (Egress)	$2.000 CLP	$24.000 CLP
Total Infraestructura	**$6.500 CLP**	**$78.000 CLP**

Tabla 4.4: Costo estimado de infraestructura híbrida (GCP Storage).

Recurso Humano	**Cantidad**	**Sueldo mensual aprox.**	**Sueldo por duración (4 meses)**
Ingeniero de Analítica Avanzada	2	$700.000 CLP	$2.800.000 CLP (c/u)
Total Desarrollo (I_0)	**2**	**$1.400.000 CLP**	**$5.600.000 CLP**

Tabla 4.5: Cálculo del costo de desarrollo (Inversión Inicial valorizada).

4.3.1. Flujo de caja

Para asegurar la viabilidad económica del proyecto, se empleará el indicador del Valor Actual Neto (VAN). La principal métrica de "Beneficio"de esta arquitectura híbrida es el **ahorro**

sustancial que genera al no utilizar hardware alquilado en la nube. Entrenar modelos de Deep Learning en instancias GPU comerciales (como Vertex AI) cuesta aproximadamente $450.000 CLP mensuales, gasto que se evita por completo al orquestar el entrenamiento en hardware local. Este ahorro ($5.400.000 CLP anuales) se contabiliza como el ingreso principal del proyecto.

Concepto	**Año 0**	**Año 1**	**Año 2**	**Año 3**	**Año 4**	**Año 5**
(+) Beneficios (Ahorro uso Cloud GPU)	$0	$5.400.000	$5.400.000	$5.400.000	$5.400.000	$5.400.000
(-) Costos Operativos (GCP Storage)	$0	($78.000)	($78.000)	($78.000)	($78.000)	($78.000)
(-) Costo Mantenimiento	$0	($400.000)	($400.000)	($400.000)	($400.000)	($400.000)
Flujo Neto (Vt)	**($5.600.000)**	**$4.922.000**	**$4.922.000**	**$4.922.000**	**$4.922.000**	**$4.922.000**

Tabla 4.6: Proyección de Flujo de Caja a 5 años (Cifras en CLP).

4.3.2. Cálculo del VAN

Para la evaluación del proyecto, se utilizará la fórmula estándar del Valor Actual Neto, como se observa en la Figura 4.1.

$$VAN = \sum_{t=1}^{n} \frac{F_t}{(1+k)^t} - I_0$$

Figura 4.1: Fórmula Matemática del Valor Actual Neto (VAN)

Donde cada uno de los términos se especifica a continuación:

Término	**Significado**
t	Intervalo de tiempo
n	Duración en años (5 años)
I_0	Inversión inicial en el periodo $t = 0$ ($5.600.000 CLP)
K	Tasa de descuento (10 %)
Vt	Flujos de caja netos obtenidos en el intervalo de tiempo t ($4.922.000 CLP)

Tabla 4.7: Términos de la fórmula de VAN aplicados al proyecto.

A continuación, se calculará el VAN con una tasa de descuento del 10 % ($K = 0{,}10$).

Año	**Valor Actual del Flujo de Caja**
Año 0 (I_0)	$-\$5{,}600{,}000/(1+0{,}10)^0 = -\$5{,}600{,}000$
Año 1	$\$4{,}922{,}000/(1+0{,}10)^1 = \$4{,}474{,}545$
Año 2	$\$4{,}922{,}000/(1+0{,}10)^2 = \$4{,}067{,}768$
Año 3	$\$4{,}922{,}000/(1+0{,}10)^3 = \$3{,}697{,}971$
Año 4	$\$4{,}922{,}000/(1+0{,}10)^4 = \$3{,}361{,}792$
Año 5	$\$4{,}922{,}000/(1+0{,}10)^5 = \$3{,}056{,}174$

Tabla 4.8: Cálculo del flujo descontado anual.

$VAN(10\,\%) = Año0 + Año1 + Año2 + Año3 + Año4 + Año5$
$VAN(10\,\%) = -5{,}600{,}000 + 4{,}474{,}545 + 4{,}067{,}768 + 3{,}697{,}971 + 3{,}361{,}792 + 3{,}056{,}174$
$VAN(10\,\%) = \$13{,}058{,}250$

En este caso, el VAN obtenido es altamente positivo (**$13.058.250 CLP**), lo que indica que el proyecto no solo es viable desde una perspectiva económica, sino que resulta ser una inversión sumamente rentable para la institución al evitar los costos prohibitivos del entrenamiento en infraestructuras comerciales de la nube.

4.4. Conclusión de factibilidad

Gracias al análisis tridimensional realizado, se concluye de forma categórica que el proyecto es **completamente factible**. A nivel técnico, se cuenta con la infraestructura local, los lenguajes y metodologías requeridas; a nivel operativo, la solución ataca un dolor directo de los investigadores mejorando sus flujos de trabajo de forma transparente; y a nivel económico, el proyecto se financia a sí mismo y genera un ahorro significativo institucional, avalado por un Valor Actual Neto positivo superior a los 13 millones de pesos chilenos.

Capítulo 5

Diseño

Introducción al capítulo.

En este capítulo se presentan los actores que interactúan con el sistema, los casos de uso identificados a partir de los requerimientos funcionales y la especificación detallada de cada uno de ellos. Además, se incluye una matriz de trazabilidad que permite relacionar los requerimientos definidos con los casos de uso correspondientes, asegurando la cobertura funcional del sistema.

5.1. Actores de Casos de Uso

En esta sección se describen los actores y entidades que interactúan dentro del alcance del proyecto, indicando sus responsabilidades y las funcionalidades a las que tienen acceso.

- **Investigador (Usuario Estándar):** Es el actor principal humano de la plataforma, enfocado en lo analítico y el ciclo de vida del *Machine Learning*. Sus responsabilidades abarcan la sincronización de datasets acústicos desde la nube, la selección de parámetros y ejecución de entrenamientos locales, la evaluación del rendimiento a través de historiales de métricas, la realización de predicciones en tiempo real y la retroalimentación del sistema validando etiquetas.
- **Administrador de Sistemas (Superusuario):** Es el actor humano encargado del soporte operativo, el mantenimiento técnico y la estabilidad del *framework* híbrido. Sus funcionalidades incluyen la gestión de cuentas y permisos de usuarios, el control y configuración de los nodos de procesamiento local, el monitoreo de la conexión a Google Cloud Platform, y la auditoría técnica mediante la revisión de registros (*logs*).
- **Sistema F.A.M.A. (Aplicación):** Actúa como la frontera del sistema y el ejecutor interno de las lógicas de negocio. Su componente principal, el Orquestador (*Backend*), es responsable de reaccionar a las peticiones de los actores humanos, descargar archivos desde Google Cloud Storage, extraer características matemáticas (espectrogramas), gestionar la memoria GPU para el entrenamiento y registrar el historial en la base de datos PostgreSQL.

5.2. Diagramas y Especificación de Casos de Uso

En la presente sección se expone el modelado del comportamiento del sistema mediante Diagramas de Casos de Uso, utilizando el Lenguaje Unificado de Modelado (UML). El objetivo de este apartado es detallar las interacciones formales entre los actores identificados (Investigador y Administrador) y el framework MLOps híbrido F.A.M.A.

Para garantizar una lectura estructurada y coherente con la arquitectura del software, los casos de uso se han categorizado en tres módulos principales, presentados en el siguiente orden lógico de ejecución: en primer lugar, el Módulo de Sesión, responsable de la autenticación y el control de acceso a la plataforma; en segundo lugar, el Módulo de Investigador, que abarca todo lo analítico y el ciclo de vida del Machine Learning (ingesta, entrenamiento y predicción). Finalmente, el Módulo de Administrador, encargado del mantenimiento operativo, gestión de nodos locales y monitoreo de los recursos en Google Cloud Platform.

A continuación se adjuntan sus respectivas tablas de especificación, las cuales describen de manera secuencial el flujo de eventos, las precondiciones y postcondiciones necesarias para dar cumplimiento a los requerimientos funcionales del sistema.

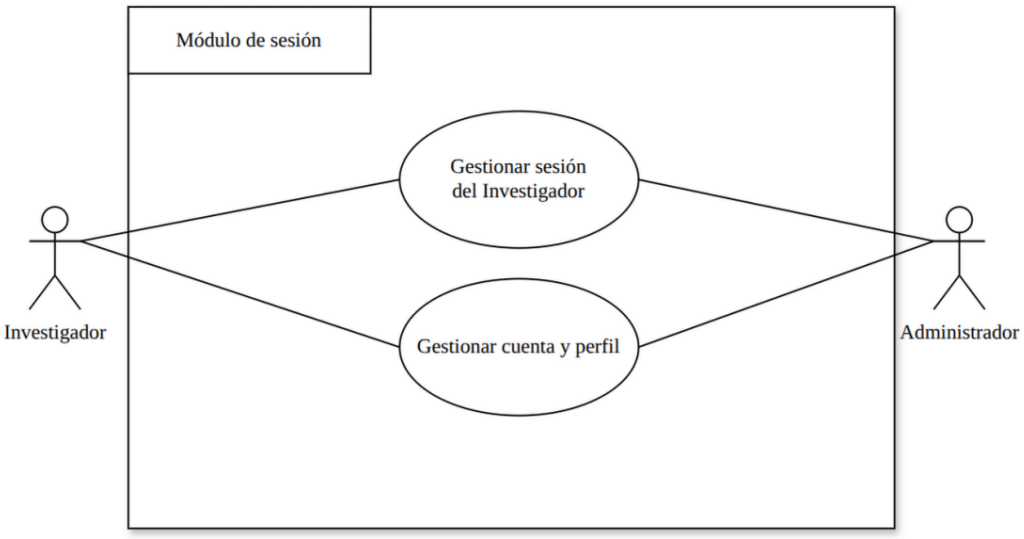


Figura 5.1: Diagrama modulo de sesión de casos de uso



Figura 5.2: Diagrama modulo de investigador de casos de uso



Figura 5.3: Diagrama modulo de administrador de casos de uso

5.2.1. Caso de Uso 1: Iniciar sesión

CU_SES_01: Iniciar Sesión	
Precondiciones: El usuario (Investigador o Administrador) debe estar previamente registrado en la base de datos PostgreSQL con sus credenciales y rol asignado.	
Actor	**Aplicación**
1) El usuario ingresa a la plataforma web y digita su correo electrónico y contraseña.	2) El sistema valida las credenciales contra los registros en la base de datos PostgreSQL.
3) El usuario visualiza el Dashboard correspondiente a su rol.	2.1) El sistema genera un token de sesión seguro y autoriza el acceso a los módulos internos.
Flujo de eventos alternativos	
Actor	**Aplicación**
1.1) El usuario ingresa una contraseña incorrecta o un correo no registrado.	2.a) El sistema rechaza el acceso y despliega un mensaje de error indicando que las credenciales son inválidas.
Postcondiciones: El usuario se encuentra autenticado y habilitado para interactuar con los procesos de la plataforma según los permisos de su perfil.	

Tabla 5.1: Especificación del Caso de Uso CU_SES_01: Iniciar Sesión

5.2.2. Caso de Uso 2: Gestionar cuenta y perfil

CU_SES_02: Gestionar Cuenta y Perfil
Descripción: Permite al usuario (Investigador o Administrador) visualizar y actualizar su información personal, modificar su contraseña de acceso y revisar el registro básico de su actividad dentro de la plataforma.
Precondiciones: El usuario debe haber iniciado sesión exitosamente en el sistema y contar con un token de autenticación válido.

Tabla 5.2: Especificación resumida del Caso de Uso CU_SES_02: Gestionar Cuenta y Perfil

5.2.3. Caso de Uso 3: Iniciar sincronización y procesamiento de datos

CU_INV_01: Iniciar Sincronización y Procesamiento de Datos	
Precondiciones: El usuario debe estar autenticado como Investigador. Deben existir datasets en el bucket de Google Cloud Storage. El entorno local debe tener conexión a internet.	
Actor	**Aplicación**
1) El usuario navega al módulo de ingesta y solicita sincronizar la base de datos. 4) El usuario visualiza la confirmación en el Dashboard.	2) El Orquestador consulta el bucket en GCP y descarga masivamente los audios crudos al entorno local. 3) El Orquestador utiliza Librosa para extraer espectrogramas y MFCC de los audios descargados. 3.1) El sistema registra los metadatos del nuevo dataset sincronizado en PostgreSQL.
Flujo de eventos alternativos	
Actor	**Aplicación**
-	2.1) Si ocurre un fallo de conexión o credenciales inválidas (JSON), el proceso se detiene con error y notifica al usuario.
Postcondiciones: Los audios están descargados localmente, convertidos en tensores matemáticos y registrados en la base de datos relacional.	

Tabla 5.3: Especificación del Caso de Uso CU_INV_01: Iniciar Sincronización y Procesamiento

5.2.4. Caso de Uso 4: Seleccionar dataset para entrenamiento

CU_INV_02: Seleccionar Dataset para Entrenamiento
Descripción: Permite al investigador visualizar a través del Dashboard el catálogo de datasets que ya han sido sincronizados y procesados por el Orquestador. El usuario elige con qué conjunto de datos desea alimentar a la red neuronal antes de iniciar la fase de modelado algorítmico.
Precondiciones: El usuario debe estar autenticado como Investigador. Debe existir al menos un registro de dataset válido y procesado (tensores) en la base de datos PostgreSQL.

Tabla 5.4: Especificación resumida del Caso de Uso CU_INV_02: Seleccionar Dataset para Entrenamiento

5.2.5. Caso de Uso 5: Entrenar modelo acústico

CU_INV_03: Entrenar Modelo Acústico	
Precondiciones: El usuario debe estar autenticado. Debe existir al menos un dataset sincronizado y procesado (tensores). Debe haber memoria GPU disponible en el entorno local.	
Actor	**Aplicación**
1) El usuario selecciona un dataset, configura los hiperparámetros (Learning Rate, Epochs) y ejecuta la acción de entrenamiento.	2) El motor local (PyTorch/TensorFlow) inicia el entrenamiento iterativo utilizando la GPU.
3) El usuario observa el reporte de métricas de precisión en el Dashboard.	2.1) Al finalizar, el sistema genera el archivo binario (.pt o .h5).
	2.2) El sistema carga el binario a Google Cloud Storage como respaldo.
	2.3) El sistema guarda las métricas (Accuracy, Loss) en PostgreSQL.
Flujo de eventos alternativos	
Actor	**Aplicación**
-	2.a) Si la memoria RAM/GPU se desborda, el sistema aborta el entrenamiento de forma segura (graceful exit) e indica el error de recursos.
Postcondiciones: Se ha generado un nuevo modelo predictivo, respaldado en la nube e indexado con sus métricas en el historial de la base de datos.	

Tabla 5.5: Especificación del Caso de Uso CU_INV_03: Entrenar Modelo Acústico

5.2.6. Caso de Uso 6: Consultar historial y métricas

CU_INV_04: Consultar Historial y Métricas
Descripción: Proporciona una interfaz gráfica donde el investigador puede auditar todos los modelos que han sido entrenados históricamente en la plataforma. Permite visualizar el rendimiento del entrenamiento, incluyendo el porcentaje de precisión (Accuracy) y la función de pérdida (Loss).
Precondiciones: El usuario debe estar autenticado. Deben existir registros de entrenamientos previos guardados en la base de datos PostgreSQL.

Tabla 5.6: Especificación resumida del Caso de Uso CU_INV_04: Consultar Historial y Métricas

5.2.7. Caso de Uso 7: Seleccionar modelo activo

CU_INV_05: Seleccionar Modelo Activo
Descripción: Permite al usuario designar explícitamente cuál de los modelos entrenados en el historial será el encargado de realizar las inferencias. Al seleccionarlo, el Orquestador carga los pesos matemáticos del archivo binario (.pt o .h5) en la memoria local, preparándolo para recibir audios de prueba.
Precondiciones: El usuario debe estar autenticado. El usuario debe haber navegado al historial de modelos y el archivo binario a seleccionar debe estar disponible e íntegro en Google Cloud Storage.

Tabla 5.7: Especificación resumida del Caso de Uso CU_INV_05: Seleccionar Modelo Activo

5.2.8. Caso de Uso 8: Realizar predicción acústica

CU_INV_06: Realizar Predicción Acústica	
Precondiciones: El usuario debe estar autenticado. Debe existir un "Modelo Activo"previamente seleccionado y cargado en memoria.	
Actor	**Aplicación**
1) El usuario carga un archivo de audio crudo de prueba a través de la interfaz web. 3) El usuario visualiza la clase detectada y el nivel de confianza.	2) El backend extrae el espectrograma del audio en tiempo real. 2.1) El Modelo Activo analiza el espectrograma (inferencia). 2.2) La aplicación despliega los resultados en pantalla en menos de 3 segundos.
Flujo de eventos alternativos	
Actor	**Aplicación**
1.1) El usuario intenta cargar un formato distinto a .wav (ej. .mp4).	1.2) El Dashboard rechaza el archivo instantáneamente advirtiendo la restricción de formato acústico.
Postcondiciones: El sistema retorna al estado de espera, listo para evaluar una nueva muestra acústica.	

Tabla 5.8: Especificación del Caso de Uso CU_INV_06: Realizar Predicción Acústica

5.2.9. Caso de Uso 9: Retroalimentar predicción errónea

CU_INV_07: Retroalimentar Predicción Errónea	
Precondiciones: El usuario debe estar autenticado como Investigador. Se debe haber ejecutado previamente una predicción acústica (*CU_INV_06*) y la interfaz debe mostrar en pantalla la etiqueta errónea arrojada por el modelo.	
Actor	**Aplicación**
1) El usuario interactúa con la interfaz web al notar un fallo en el análisis y presiona el botón para corregir la predicción.	2) El Dashboard despliega un panel interactivo de telemetría con el catálogo de clases válidas del sistema.
3) El usuario selecciona la etiqueta real del audio desde el menú desplegable y confirma la acción.	4) El backend captura los metadatos (*IE_04*), actualiza el estado en PostgreSQL y transfiere de forma segura el audio junto con su nueva etiqueta al repositorio dinámico en Google Cloud Storage. 4.1) El sistema notifica el éxito del registro y limpia la interfaz.
Flujo de eventos alternativos	
Actor	**Aplicación**
1.1) El usuario cancela la operación antes de enviar la corrección.	1.2) El sistema cierra el panel de retroalimentación y regresa a la pantalla de visualización estándar sin alterar los datos.
Postcondiciones: El audio mal clasificado queda indexado con su etiqueta real en la base de datos relacional y respaldado en la nube, quedando disponible para el próximo ciclo de re-entrenamiento.	

Tabla 5.9: Especificación del Caso de Uso CU_INV_07: Retroalimentar Predicción Errónea

5.2.10. Caso de Uso 10: Gestionar usuarios

CU_ADM_01: Gestionar usuarios	
Precondiciones: El usuario debe estar autenticado en la plataforma bajo el rol de Administrador de Sistemas.	
Actor	**Aplicación**
1) El administrador accede al panel de gestión de usuarios y selecciona la opción para crear, editar o deshabilitar una cuenta de Investigador.	2) El sistema despliega el listado actual de usuarios registrados consultando la base de datos PostgreSQL.
3) El administrador completa el formulario con los datos y el nivel de acceso, y confirma la acción.	4) El sistema valida la información, actualiza los registros en la base de datos y confirma el éxito de la operación en pantalla.
Flujo de eventos alternativos	
Actor	**Aplicación**
3.1) El administrador intenta registrar un usuario con un correo electrónico ya existente.	4.1) El sistema detiene el proceso y alerta que el identificador (correo) ya se encuentra registrado en PostgreSQL.
Postcondiciones: El catálogo de usuarios e investigadores queda actualizado, impactando inmediatamente los permisos de acceso a la plataforma F.A.M.A.	

Tabla 5.10: Especificación del Caso de Uso CU_ADM_01: Gestionar usuarios

5.2.11. Caso de Uso 11: Gestionar nodos de procesamiento local

CU_ADM_02: Gestionar nodos de procesamiento local
Descripción: Permite al administrador configurar y auditar los recursos de hardware del entorno local (máquina con GPU). Incluye la verificación de disponibilidad de memoria RAM y el estado de los controladores (CUDA) necesarios para que el Orquestador ejecute los entrenamientos matemáticos.
Precondiciones: El usuario debe estar autenticado como Administrador. El nodo de procesamiento debe estar encendido y conectado a la red local.

Tabla 5.11: Especificación resumida del Caso de Uso CU_ADM_02: Gestionar nodos de procesamiento local

5.2.12. Caso de Uso 12: Gestionar recursos cloud

CU_ADM_03: Gestionar recursos cloud
Descripción: Proporciona herramientas para auditar la conexión con Google Cloud Platform. Permite al administrador actualizar el archivo de credenciales IAM (JSON) de manera segura mediante variables de entorno y revisar métricas de almacenamiento general del bucket en la nube.
Precondiciones: El usuario debe estar autenticado como Administrador. Debe existir conexión activa a internet para consultar las APIs de GCP.

Tabla 5.12: Especificación resumida del Caso de Uso CU_ADM_03: Gestionar recursos cloud

5.2.13. Caso de Uso 13: Monitorear pipelines y registros

CU_ADM_04: Monitorear pipelines y registros	
Precondiciones: El usuario debe estar autenticado como Administrador. Deben existir operaciones previas en el sistema que hayan generado archivos de registro (.log) o entradas en la base de datos.	
Actor	**Aplicación**
1) El administrador navega a la sección de auditoría del sistema y solicita visualizar los registros de sincronización y entrenamiento.	2) El sistema consulta los archivos locales (.log) y la base de datos PostgreSQL, desplegando un historial cronológico de los procesos ejecutados.
3) El administrador selecciona un registro específico para auditar posibles fallas de integración.	4) El sistema formatea y presenta los detalles técnicos de la transacción (ej. cantidad de bytes leídos desde la nube, tiempos de ejecución y errores).
Flujo de eventos alternativos	
Actor	**Aplicación**
-	2.1) Si los archivos de registro (.log) han sido eliminados o están corruptos, el sistema muestra el estado "No disponible"para ese registro en particular.
Postcondiciones: El administrador obtiene visibilidad completa sobre el rendimiento y posibles errores del flujo MLOps, facilitando el soporte técnico.	

Tabla 5.13: Especificación del Caso de Uso CU_ADM_04: Monitorear pipelines y registros

5.2.14. Caso de Uso 14: Controlar ejecución de procesos

CU_ADM_05: Controlar ejecución de procesos
Descripción: Otorga al administrador la capacidad de interrumpir forzosamente (kill process) operaciones en segundo plano, como ingestas masivas atrapadas en bucle o entrenamientos de redes neuronales que estén generando un desbordamiento de memoria (Out of Memory), garantizando la estabilidad del nodo local.
Precondiciones: El usuario debe estar autenticado como Administrador. Debe existir al menos un proceso gestionado por el Orquestador ejecutándose en segundo plano.

Tabla 5.14: Especificación resumida del Caso de Uso CU_ADM_05: Controlar ejecución de procesos

5.3. Matriz de Trazabilidad

La matriz de trazabilidad permite relacionar los requerimientos funcionales con los casos de uso definidos para verificar la cobertura de las funcionalidades del sistema.

ID Req.	Descripción del Requerimiento Funcional	Casos de Uso Asociados
RF_01	Autenticación automática del Orquestador con Google Cloud Storage mediante archivo JSON seguro.	**CU_INV_01** (Sincronización) **CU_ADM_03** (Recursos Cloud)
RF_02	Descarga masiva de datasets de audio crudos desde la nube al almacenamiento local.	**CU_INV_01** (Sincronización)
RF_03	Extracción automatizada de características matemáticas (espectrogramas y MFCC) de los audios.	**CU_INV_01** (Sincronización) **CU_INV_06** (Predicción)
RF_04	Entrenamiento local de la red neuronal mediante hardware con aceleración gráfica (GPU).	**CU_INV_02** (Elegir Dataset) **CU_INV_03** (Entrenamiento) **CU_INV_04** (Consultar Historial)
RF_05	Visualización interactiva de predicciones acústicas (clase y confianza) en tiempo real mediante un Dashboard.	**CU_INV_05** (Elegir Modelo) **CU_INV_06** (Predicción)
RF_06	Registro de telemetría y retroalimentación activa sobre la exactitud de las predicciones a través del Dashboard para el almacenamiento de datos corregidos.	**CU_INV_07** (Retroalimentar Predicción)

Tabla 5.15: Matriz de trazabilidad: Requerimientos Funcionales - Casos de Uso

Capítulo 6

Desarrollo del Trabajo

El presente capítulo tiene por objetivo detallar la estructuración técnica del framework MLOps híbrido F.A.M.A. Se describen los servicios web necesarios para la orquestación nube-local, el modelo de datos relacional que garantiza la trazabilidad de los experimentos, y el diseño visual de la interfaz de usuario. Finalmente, se expone la arquitectura del sistema, materializando los requerimientos previos en una solución cohesiva y escalable.

6.1. Descripción de los servicios web

Para el funcionamiento de la arquitectura híbrida de F.A.M.A., se requiere la implementación y consumo de los siguientes servicios web:

- **API de Google Cloud Storage:** Servicio externo necesario para la ingesta masiva de archivos de audio (*datasets*) y la subida de los modelos binarios entrenados. Se consume mediante la librería oficial de Google usando credenciales IAM en formato `JSON`.
- **Servicio de Base de Datos (Cloud SQL / PostgreSQL):** Conexión transaccional para el almacenamiento del historial de métricas, usuarios y registros de retroalimentación activa.
- **API REST Interna (Orquestador Backend):** Servicio local (construido con `FastAPI`) que expone *endpoints* para que el *Dashboard* (*Frontend*) pueda gatillar la extracción de espectrogramas, iniciar el entrenamiento tensorial en la GPU y solicitar inferencias predictivas en tiempo real.

6.1.1. Diagrama del proyecto

El flujo operativo y la interconexión de los componentes del proyecto se representan mediante una arquitectura de servicios centralizada. La interacción se inicia en la capa de presentación (usuario web), que se comunica mediante peticiones REST con la lógica del orquestador backend local. Este orquestador actúa como coordinador del sistema, encargándose de descargar los lotes de datos desde Google Cloud Storage, delegar la extracción matemática y el procesamiento al

hardware local (GPU NVIDIA) y, finalmente, almacenar las métricas y los registros resultantes en la base de datos relacional (Cloud SQL).

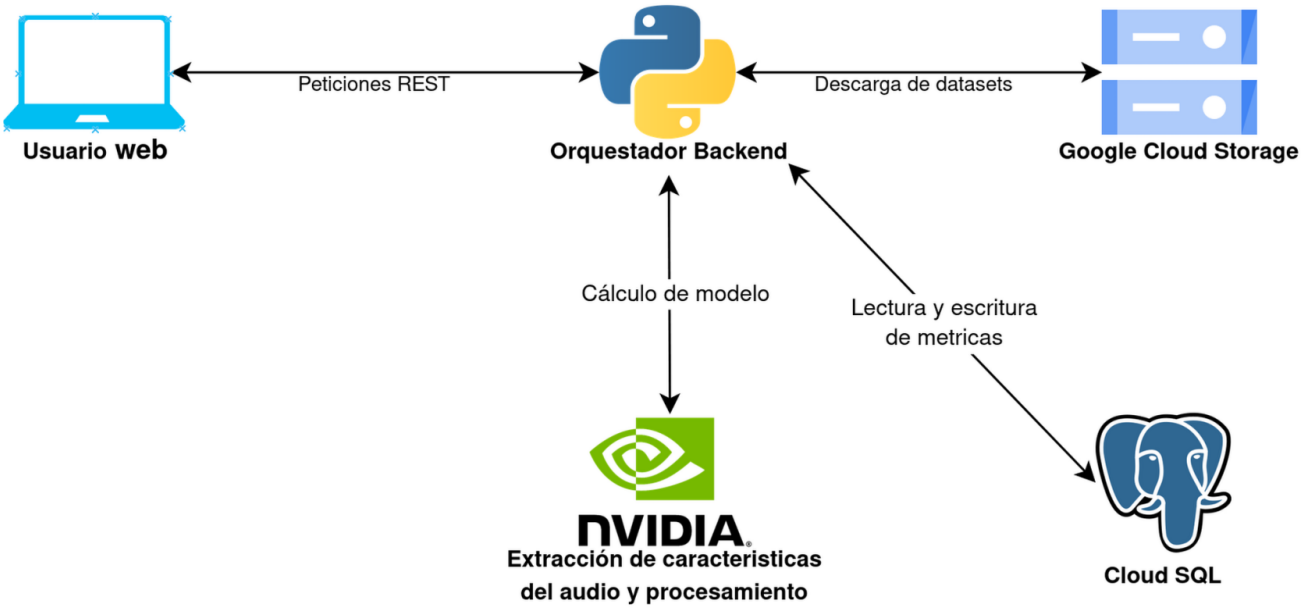


Figura 6.1: Diagrama de servicios web e interconexión operativa del proyecto.

6.2. Modelo de datos

6.2.1. Esquema de base de datos

Se presenta el esquema físico de la base de datos, indicando tablas, tipos de datos, claves primarias y foráneas, así como restricciones de integridad.

Tipo ENUM	**Valores permitidos**
rol_usuario	Ínvestigador', Ádministrador'
estado_conjunto_datos	'pendiente', 'procesado', érror'
estado_modelo	éntrenando', 'listo', 'fallido'
tipo_registro	íngesta', éntrenamiento', 'prediccion', 'retroalimentacion', ádministracion'
estado_registro	én_curso', éxito', érror'
estado_nodo	áctivo', ínactivo', 'mantenimiento', érror'

Tabla 6.1: Tipos enumerados (ENUMs)

Tabla usuario

Columna	Tipo de dato	Restricciones
id_usuario	SERIAL	PRIMARY KEY
correo_electronico	VARCHAR(150)	NOT NULL UNIQUE
hash_contrasena	VARCHAR(255)	NOT NULL
rol	rol_usuario	NOT NULL
activo	BOOLEAN	NOT NULL DEFAULT TRUE
ultimo_acceso	TIMESTAMP	Opcional
fecha_creacion	TIMESTAMP	NOT NULL DEFAULT NOW()

Tabla 6.2: Tabla usuario

Tabla nodo_procesamiento

Columna	Tipo de dato	Restricciones
id_nodo	SERIAL	PRIMARY KEY
nombre	VARCHAR(100)	NOT NULL UNIQUE
direccion_ip	VARCHAR(45)	Opcional
gpu_modelo	VARCHAR(150)	Opcional
memoria_ram_mb	INT	Opcional
cuda_disponible	BOOLEAN	Opcional
estado	estado_nodo	NOT NULL DEFAULT áctivo’
ultimo_chequeo	TIMESTAMP	NOT NULL DEFAULT NOW()

Tabla 6.3: Tabla nodo_procesamiento

Tabla conjunto_datos

Columna	Tipo de dato	Restricciones
id_conjunto_datos	SERIAL	PRIMARY KEY
nombre	VARCHAR(100)	NOT NULL
ruta_gcp	VARCHAR(500)	NOT NULL
estado	estado_conjunto_datos	NOT NULL
id_usuario_carga	INT	NOT NULL FK → usuario ON DELETE RESTRICT
fecha_carga	TIMESTAMP	NOT NULL DEFAULT NOW()
tamano_total_bytes	BIGINT	NOT NULL DEFAULT 0
cantidad_audios	INT	NOT NULL DEFAULT 0
finalidad	TEXT	NOT NULL
base_licitud	VARCHAR(50)	NOT NULL

Tabla 6.4: Tabla conjunto_datos

Tabla audio

Columna	Tipo de dato	Restricciones
id_audio	SERIAL	PRIMARY KEY
id_conjunto_datos	INT	NOT NULL FK → conjunto_datos ON DELETE CASCADE
nombre_archivo	VARCHAR(255)	NOT NULL
ruta_gcp	VARCHAR(500)	NOT NULL
clase	VARCHAR(100)	NOT NULL
frecuencia_muestreo	INT	Opcional
duracion_segundos	DECIMAL(10,3)	Opcional
tamano_bytes	BIGINT	NOT NULL DEFAULT 0
hash_archivo	VARCHAR(64)	Opcional
fecha_subida	TIMESTAMP	NOT NULL DEFAULT NOW()

Tabla 6.5: Tabla audio

Tabla modelo

Columna	Tipo de dato	Restricciones
id_modelo	SERIAL	PRIMARY KEY
id_usuario	INT	NOT NULL FK → usuario ON DELETE RESTRICT
id_conjunto_datos	INT	NOT NULL FK → conjunto_datos ON DELETE RESTRICT
clase_objetivo	VARCHAR(100)	NOT NULL
actualizacion_de	INT	FK → modelo ON DELETE SET NULL
arquitectura	VARCHAR(255)	NOT NULL
epocas	INT	NOT NULL
tasa_aprendizaje	DECIMAL(10,6)	NOT NULL
tamano_lote	INT	NOT NULL
precision	DECIMAL(6,4)	Opcional
perdida	DECIMAL(6,4)	Opcional
ruta_binario_gcp	VARCHAR(500)	NOT NULL
tamano_bytes	BIGINT	NOT NULL DEFAULT 0
hash_binario	VARCHAR(64)	Opcional
activo	BOOLEAN	NOT NULL DEFAULT FALSE
estado	estado_modelo	NOT NULL
fecha_entrenamiento	TIMESTAMP	NOT NULL DEFAULT NOW()

Tabla 6.6: Tabla modelo

Tabla metrica_entrenamiento

Columna	Tipo de dato	Restricciones
id_metrica	SERIAL	PRIMARY KEY
id_modelo	INT	NOT NULL FK → modelo ON DELETE CASCADE
epoca	INT	NOT NULL
precision	DECIMAL(6,4)	NOT NULL
perdida	DECIMAL(6,4)	NOT NULL
puntuacion_validacion	DECIMAL(6,4)	Opcional
tiempo_epoca	DECIMAL(10,4)	Opcional

Tabla 6.7: Tabla metrica_entrenamiento

Tabla prediccion

Columna	Tipo de dato	Restricciones
id_prediccion	SERIAL	PRIMARY KEY
id_modelo	INT	NOT NULL FK → modelo ON DELETE RESTRICT
id_usuario	INT	NOT NULL FK → usuario ON DELETE RESTRICT
id_audio	INT	FK → audio (exclusivo con ruta_audio_prueba)
ruta_audio_prueba	VARCHAR(500)	NULL (exclusivo con id_audio)
tamano_bytes	BIGINT	NOT NULL DEFAULT 0
hash_archivo	VARCHAR(64)	Opcional
etiqueta_predicha	VARCHAR(100)	NOT NULL
confianza	DECIMAL(5,4)	NOT NULL; CHECK BETWEEN 0 AND 1
fecha_prediccion	TIMESTAMP	NOT NULL DEFAULT NOW()

Tabla 6.8: Tabla prediccion

Tabla retroalimentacion

Columna	Tipo de dato	Restricciones
id_retroalimentacion	SERIAL	PRIMARY KEY
id_prediccion	INT	NOT NULL UNIQUE FK → prediccion ON DELETE RESTRICT
id_usuario	INT	NOT NULL FK → usuario ON DELETE RESTRICT
etiqueta_corregida	VARCHAR(100)	NULL; ver CHECK de integridad
fue_correcta	BOOLEAN	NOT NULL
procesado	BOOLEAN	NOT NULL DEFAULT FALSE
fecha_retroalimentacion	TIMESTAMP	NOT NULL DEFAULT NOW()

Tabla 6.9: Tabla retroalimentacion

Tabla registro_pipeline

Columna	Tipo de dato	Restricciones
id_registro	SERIAL	PRIMARY KEY
id_usuario	INT	FK → usuario ON DELETE RESTRICT
id_nodo	INT	FK → nodo_procesamiento ON DELETE RESTRICT
tipo	tipo_registro	NOT NULL
estado	estado_registro	NOT NULL
detalle	TEXT	Opcional
archivos_descargados	INT	Opcional
bytes_leidos	BIGINT	Opcional
fecha_inicio	TIMESTAMP	NOT NULL DEFAULT NOW()
fecha_fin	TIMESTAMP	Opcional

Tabla 6.10: Tabla registro_pipeline

6.2.2. Entidad-Relación

El diagrama Entidad-Relación (E-R) constituye la representación conceptual y abstracta de la información que circulará a través del framework F.A.M.A. A diferencia del esquema físico, este modelo se centra en las reglas de negocio, identificando las entidades esenciales, sus atributos semánticos y las restricciones de cardinalidad que gobiernan el ciclo de vida MLOps sin descender a definiciones de implementación de software o tipos de datos específicos.

A continuación, se describen las entidades nucleares identificadas para el correcto funcionamiento de la arquitectura híbrida:

- **Usuario**: Entidad que representa a los investigadores y administradores. Sus atributos conceptuales son el correo electrónico identificador, sus credenciales de acceso y el rol asignado que determina sus privilegios operativos.
- **Conjunto de Datos**: Representa los lotes estáticos de señales acústicas almacenados de manera centralizada en la nube. Se caracteriza por su nombre, finalidad de la investigación y metadatos de volumen.
- **Audio**: Muestra las muestras acústicas individuales. Contiene atributos como el nombre del archivo, su clase o etiqueta biológica original, y sus propiedades físicas de duración y frecuencia de muestreo.
- **Modelo**: Entidad analítica que encarna los pesos matemáticos de la red neuronal entrenada localmente. Almacena las configuraciones de hiperparámetros (tasa de aprendizaje, épocas, tamaño de lote) y los indicadores macro de rendimiento técnico.
- **Métrica de Entrenamiento**: Entidad dependiente que registra de forma secuencial la evolución matemática del modelo (precisión y pérdida) desglosada por cada época de ejecución.

- **Predicción**: Documenta la inferencia en tiempo real realizada por un modelo activo sobre una muestra acústica de prueba, registrando la etiqueta predicha y el nivel de confianza algorítmico.
- **Retroalimentación**: Elemento crítico que materializa el ciclo de aprendizaje continuo sugerido. Captura la acción del investigador al validar o corregir una inferencia de la Inteligencia Artificial, enlazando el error con la etiqueta real correcta.
- **Nodo de Procesamiento**: Representa conceptualmente los recursos de hardware locales disponibles para la computación (capacidad de memoria y disponibilidad de aceleración por GPU).
- **Registro de Pipeline**: Componente orientado a la auditoría técnica y trazabilidad, encargado de registrar las marcas de tiempo y volumen de datos de cada proceso transaccional nube-local.

En la Figura 6.2 se ilustra el diagrama conceptual resultante, modelando de manera gráfica las interacciones y dependencias mutuas de las entidades descritas.

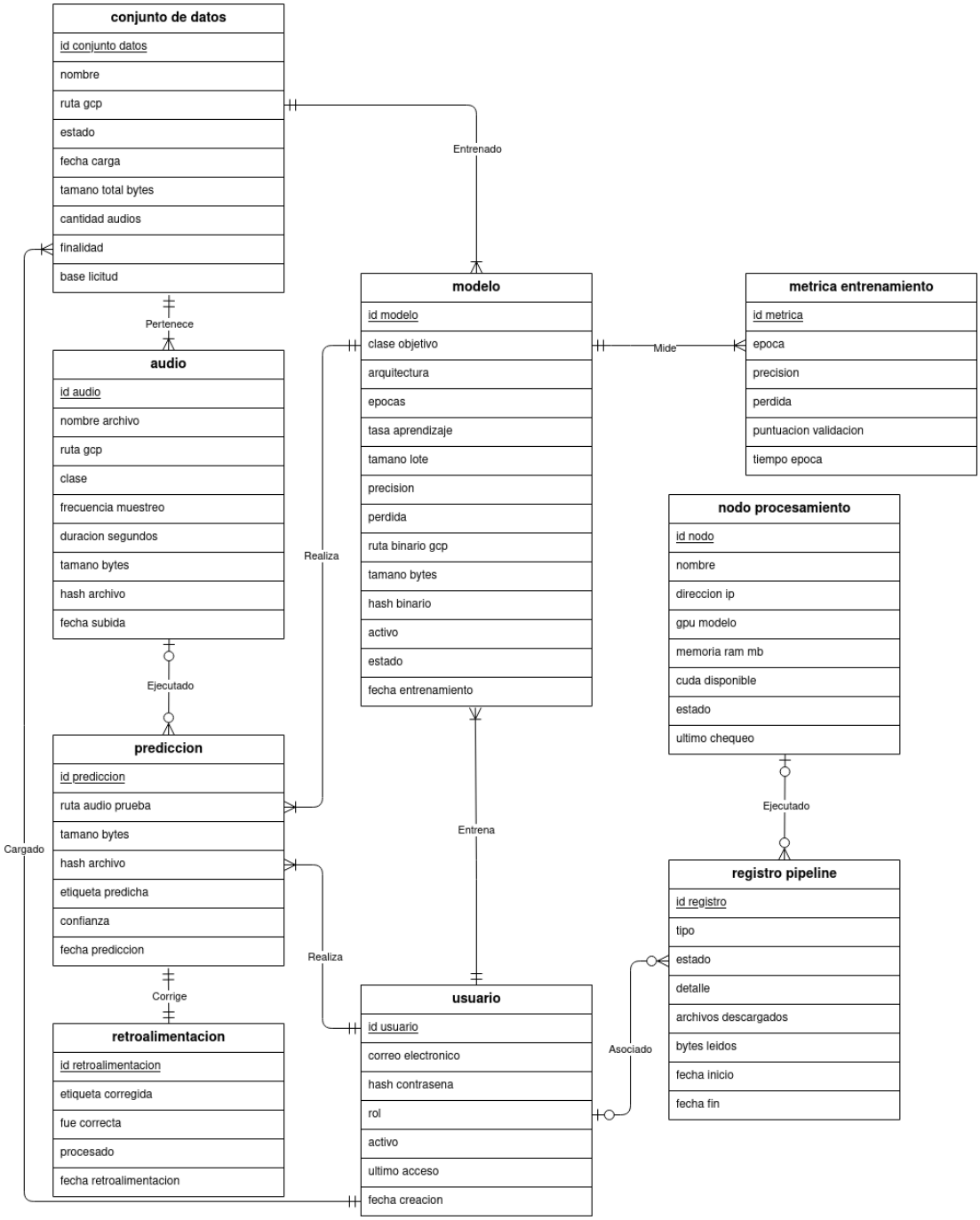


Figura 6.2: Diagrama Entidad-Relación conceptual del sistema F.A.M.A.

6.2.3. Modelo-Relación

A partir del esquema definido anteriormente, se deriva el modelo relacional, representado mediante relaciones normalizadas que eliminan redundancias, aseguran la integridad referencial y contemplan la trazabilidad del consentimiento de los datos.

Relación	**Atributos y Claves**
usuario	id_usuario, correo_electronico, hash_contrasena, rol, activo, ultimo_acceso, fecha_creacion
nodo_procesamiento	id_nodo, nombre, direccion_ip, gpu_modelo, memoria_ram_mb, cuda_disponible, estado, ultimo_chequeo
conjunto_datos	id_conjunto_datos, nombre, ruta_gcp, estado, *id_usuario_carga (FK)*, fecha_carga, tamano_total_bytes, cantidad_audios, finalidad, base_licitud
audio	id_audio, *id_conjunto_datos (FK)*, nombre_archivo, ruta_gcp, clase, frecuencia_muestreo, duracion_segundos, tamano_bytes, hash_archivo, fecha_subida
modelo	id_modelo, *id_usuario (FK)*, *id_conjunto_datos (FK)*, clase_objetivo, *actualizacion_de (FK)*, arquitectura, epocas, tasa_aprendizaje, tamano_lote, precision, perdida, ruta_binario_gcp, tamano_bytes, hash_binario, activo, estado, fecha_entrenamiento
metrica_entrenamiento	id_metrica, *id_modelo (FK)*, epoca, precision, perdida, puntuacion_validacion, tiempo_epoca
prediccion	id_prediccion, *id_modelo (FK)*, *id_usuario (FK)*, *id_audio (FK)*, ruta_audio_prueba, tamano_bytes, hash_archivo, etiqueta_predicha, confianza, fecha_prediccion
retroalimentacion	id_retroalimentacion, *id_prediccion (FK)*, *id_usuario (FK)*, etiqueta_corregida, fue_correcta, procesado, fecha_retroalimentacion
registro_pipeline	id_registro, *id_usuario (FK)*, *id_nodo (FK)*, tipo, estado, detalle, archivos_descargados, bytes_leidos, fecha_inicio, fecha_fin

Tabla 6.11: Modelo Relacional Normalizado de F.A.M.A.

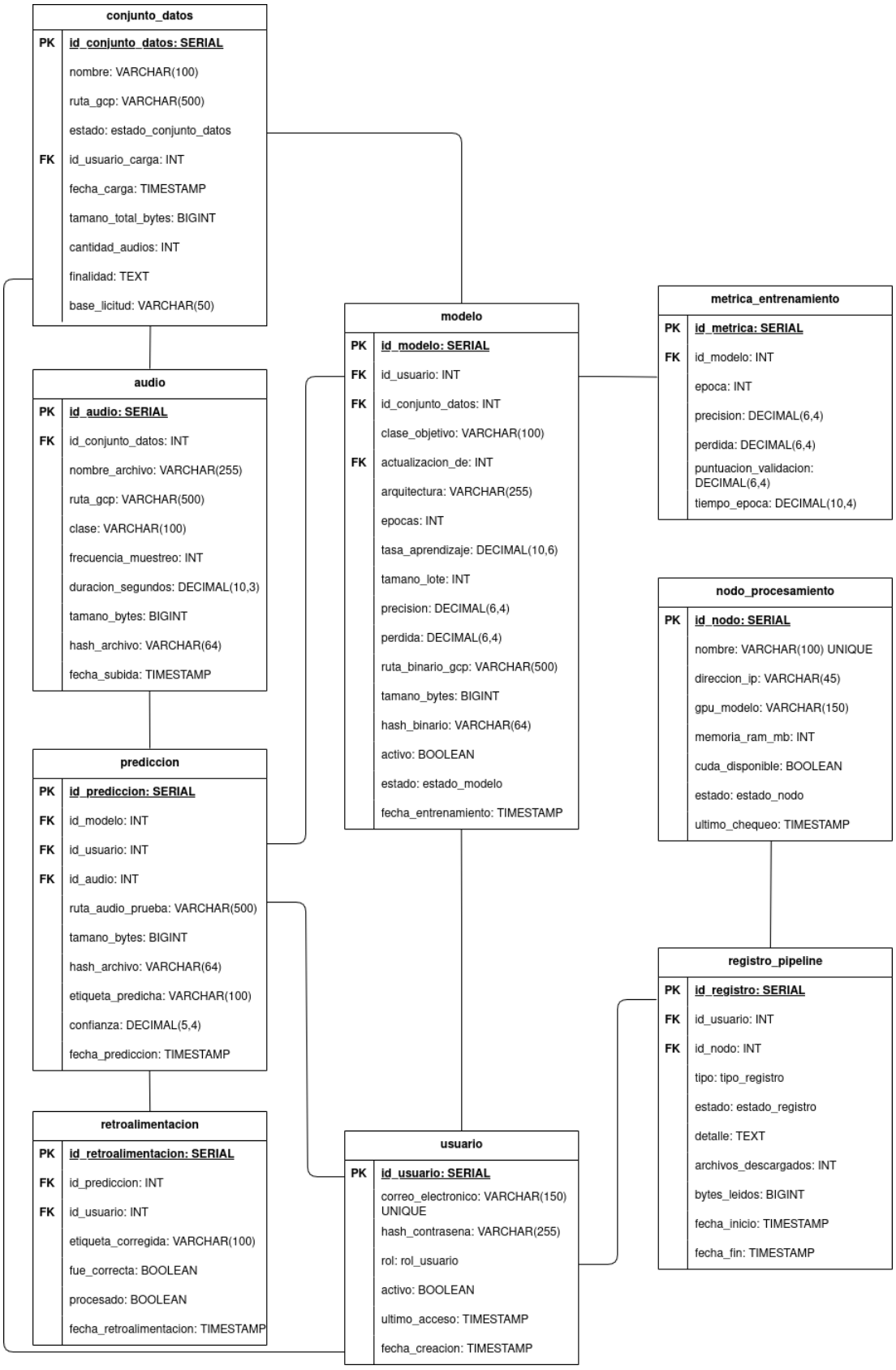


Figura 6.3: Diagrama del Modelo Relacional de la base de datos.

6.3. Diseño de interfaz y navegación

6.3.1. Guías de estilo

Se establecen los lineamientos visuales y de interacción que garantizan la coherencia en toda la aplicación web. Al tratarse de un *framework* científico enfocado en el análisis acústico, se prioriza un diseño minimalista, funcional y orientado a los datos (*Data-Driven Design*). Las guías de estilo de F.A.M.A. se fundamentan en los siguientes principios operativos:

- **Tema y Contraste:** Se implementará un modo oscuro (*Dark Mode*) nativo. Esta decisión técnica tiene el propósito de reducir la fatiga visual de los investigadores durante sesiones analíticas prolongadas, permitiendo además un alto nivel de contraste para la correcta visualización matemática de los espectrogramas renderizados en pantalla.
- **Paleta Semántica de Colores:** Para mantener la atención en los datos, la interfaz utilizará una escala de grises para sus componentes estructurales. El uso de colores saturados se reservará estrictamente para la retroalimentación del sistema: tonos verdes para indicar entrenamientos exitosos o ingestas completadas, y tonos rojos/naranjas para alertar sobre errores de desbordamiento de memoria (OOM) o para marcar clasificaciones erróneas en el ciclo de telemetría.
- **Disposición y Componentes:** La estructura visual adoptará un sistema de cuadrícula (*Grid System*) responsivo. Se maximizará el lienzo central para el despliegue del *Dashboard* interactivo, mientras que las herramientas de orquestación (selección de *datasets*, configuración de hiperparámetros) se encapsularán en menús laterales colapsables para no saturar la carga cognitiva del usuario.

6.3.2. Logotipo

El logotipo de F.A.M.A. adopta un enfoque estrictamente tipográfico y minimalista, alineado con la estética técnica y orientada a los datos del *framework*. Su construcción visual se estructura en dos niveles jerárquicos que forman un bloque geométrico equilibrado:

- **Marca principal:** Las siglas de F.A.M.A. se presentan en gran escala utilizando una tipografía sin remates (*sans-serif*) de peso grueso (*Bold*). Esto transmite la solidez, escalabilidad y estabilidad que caracterizan a la arquitectura del orquestador de *Machine Learning*.
- **Bajada descriptiva:** Justo debajo de la sigla principal, se ubica el nombre extendido del proyecto en un tamaño de fuente significativamente menor. Este texto secundario cuenta con un espaciado entre letras (*tracking*) ajustado matemáticamente para que sus bordes se alineen a la perfección con la anchura total de las siglas superiores, creando una composición rectangular compacta.

Esta decisión de diseño prescinde de isotipos complejos o distractores visuales, asegurando que el logotipo sea altamente legible tanto en el modo oscuro del *Dashboard* como en los reportes en formato PDF, manteniendo su integridad escalar en cualquier dispositivo.



Figura 6.4: Diseño tipográfico y composición del logotipo de F.A.M.A.

6.3.3. Guía de colores

El diseño visual de la aplicación web está pensado para ser funcional, priorizando la legibilidad de los datos y reduciendo la fatiga visual del usuario. Para lograr esto, el sistema utiliza por defecto un modo oscuro (*dark mode*), el cual genera un alto contraste con las gráficas y los textos, facilitando las sesiones prolongadas de análisis acústico.

Paleta de colores principal

La interfaz no utiliza un color corporativo tradicional que predomine en toda la pantalla, sino que emplea una escala de grises para su estructura básica y reserva los colores vivos exclusivamente para comunicar el estado del sistema.

A continuación, se detalla la paleta de colores base utilizada en los distintos elementos de la plataforma:

Color	**Elemento visual**	**Propósito en la plataforma**
Gris muy oscuro	Fondo de página	Actúa como lienzo principal para no cansar la vista del usuario.
Blanco	Texto principal	Asegura la máxima legibilidad de los títulos y datos.
Gris oscuro	Tarjetas y paneles	Agrupa visualmente la información y separa los distintos módulos.
Gris claro	Botones principales	Destaca las acciones más importantes (ej. “Entrenar modelo”, “Sincronizar”).
Gris medio	Botones secundarios	Para acciones de menor importancia o elementos inactivos.
Negro carbón	Consola de sistema	Simula una terminal clásica para leer los registros de actividad.

Tabla 6.12: Paleta de colores estructural de la interfaz de usuario.

Reglas semánticas (El uso del color para comunicar)

Para que el usuario comprenda rápidamente qué está ocurriendo en el sistema sin necesidad de leer textos largos, F.A.M.A. utiliza un semáforo de colores universal. Cada color vivo tiene un significado estricto que se respeta en todas las pantallas (Dashboard, Ingesta, Entrenamiento y Predicción):

Color	Significado	Contexto de uso
Verde	Éxito / Activo	Indica que un proceso terminó correctamente, que la Inteligencia Artificial tiene un nivel de confianza alto (mayor al 80 %) o que los componentes (como la GPU o la Nube) están conectados.
Rojo	Error / Alerta	Se utiliza para advertir fallos críticos, errores al cargar archivos, detenciones forzadas o un nivel de precisión muy bajo del modelo.
Amarillo	Precaución	Señala advertencias preventivas o indica que el modelo tiene un nivel de confianza dudoso (entre 50 % y 80 %).
Azul	Información	Se usa para mostrar datos neutrales, indicar procesos en curso y para dibujar las curvas de rendimiento en los gráficos.

Tabla 6.13: Uso semántico del color para la comunicación de estados.

Colores en las representaciones gráficas

Los gráficos matemáticos que muestran el rendimiento de la Inteligencia Artificial son el núcleo visual del proyecto. Para evitar confusiones, las líneas y ondas acústicas siempre utilizan los mismos colores:

- **Línea Azul:** Representa la **Precisión (*Accuracy*)**. Muestra visualmente qué tan bien está aprendiendo el modelo. También se usa para dibujar la forma de onda de los audios analizados.
- **Línea Roja:** Representa la **Pérdida (*Loss*)**. Ilustra el margen de error del modelo durante su entrenamiento, asociando el color rojo a un factor que el investigador debe intentar disminuir.
- **Líneas Blancas (semitransparentes):** Forman la cuadrícula de fondo de los gráficos para facilitar la lectura de los valores numéricos sin distraer la atención de las curvas principales.

Coherencia visual entre módulos

Esta guía de colores se aplica de manera idéntica en todas las vistas del sistema. Esto significa que si el usuario aprende que una etiqueta verde significa “éxito” en el panel de Ingesta, esa misma etiqueta verde significará lo mismo en los módulos de Entrenamiento y Predicción.

Esta estandarización reduce la curva de aprendizaje y permite que el investigador se enfoque puramente en el análisis de los datos acústicos.

6.3.4. Tipografía

La selección tipográfica para la interfaz de F.A.M.A. se basó en los principios de legibilidad técnica y jerarquía visual de la información. Al ser un entorno científico donde se monitorean métricas decimales en tiempo real (como precisión, pérdida algorítmica y progreso por épocas), es vital evitar cualquier ambigüedad en los caracteres. Para ello, se han definido tres familias tipográficas específicas:

- **Primaria (Montserrat):** Tipografía geométrica *sans-serif* utilizada para títulos y encabezados. Su peso y estructura aportan jerarquía y rápida escaneabilidad en la navegación del *Dashboard*.
- **Secundaria (Open Sans):** Tipografía humanista *sans-serif* optimizada para alta legibilidad en pantallas. Se emplea para el cuerpo de texto, descripciones y componentes interactivos (formularios, botones desplegables).
- **Monoespaciada (Fira Code):** Fuente técnica reservada exclusivamente para la visualización de *logs* del orquestador, identificadores de la base de datos (IDs) y despliegue de métricas matemáticas, garantizando que cada carácter (número o letra) ocupe exactamente el mismo ancho visual en pantalla.

En la Tabla 6.14 se detallan las especificaciones, tamaños y pesos aplicados a los distintos elementos estructurales de la interfaz de usuario.

Elemento UI	**Fuente, Peso y Tamaño**	**Contexto de Uso**
Títulos de Módulo	Montserrat Bold, 24px	Encabezados principales de las vistas del *Dashboard*.
Subtítulos	Montserrat SemiBold, 18px	Títulos de gráficos, modales y tarjetas (*Cards*) de métricas.
Cuerpo de Texto	Open Sans Regular, 14px	Párrafos descriptivos, base de licitud y datos dentro de tablas.
Controles UI	Open Sans Medium, 12px	Botones, pestañas de navegación y menús laterales colapsables.
Consola y Métricas	Fira Code Regular, 13px	Consola de telemetría, progreso de *epochs* y precisión del modelo.

Tabla 6.14: Especificaciones tipográficas de la interfaz web de F.A.M.A.

6.3.5. Composición de interfaces

Se presentan los mockups o prototipos de las pantallas principales, indicando la disposición de componentes, jerarquía visual y flujo de navegación entre vistas.

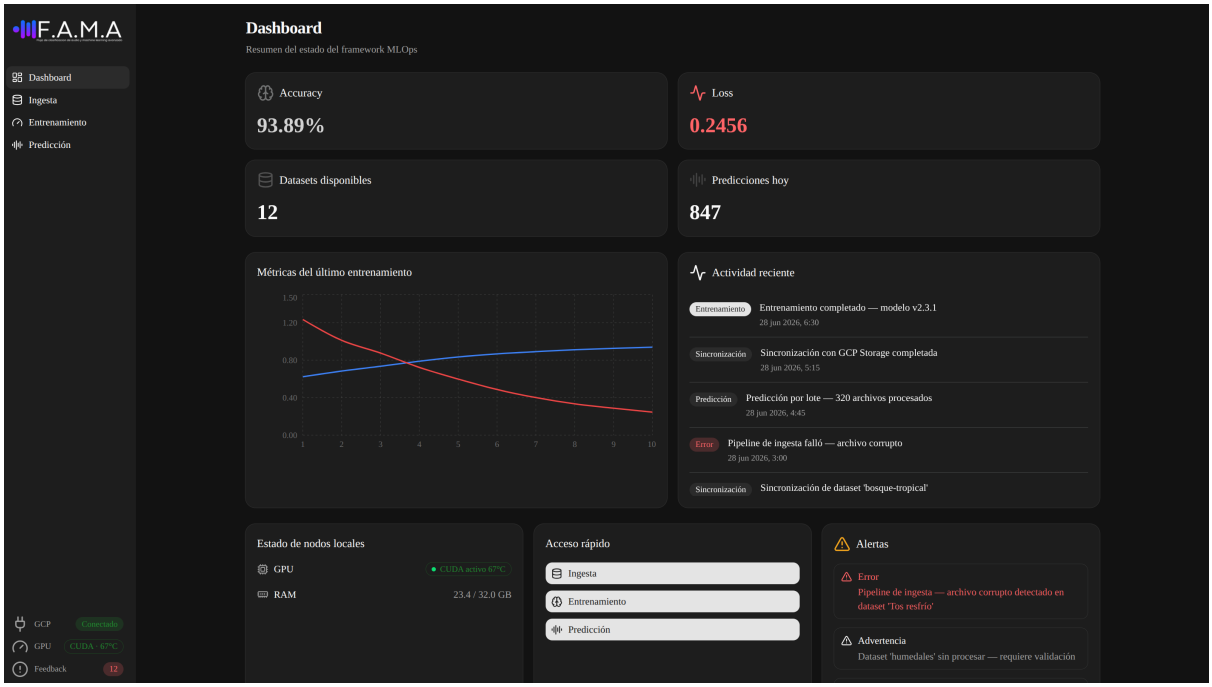


Figura 6.5: Vista del Dashboard

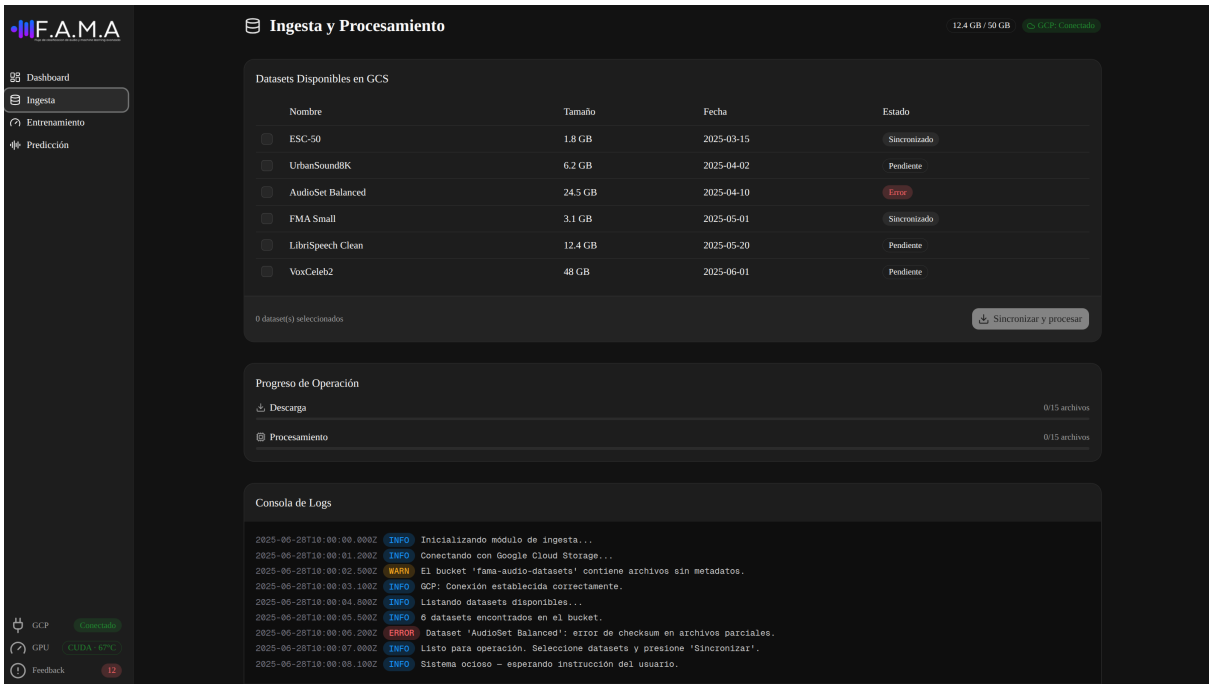


Figura 6.6: Vista de ingesta de datos

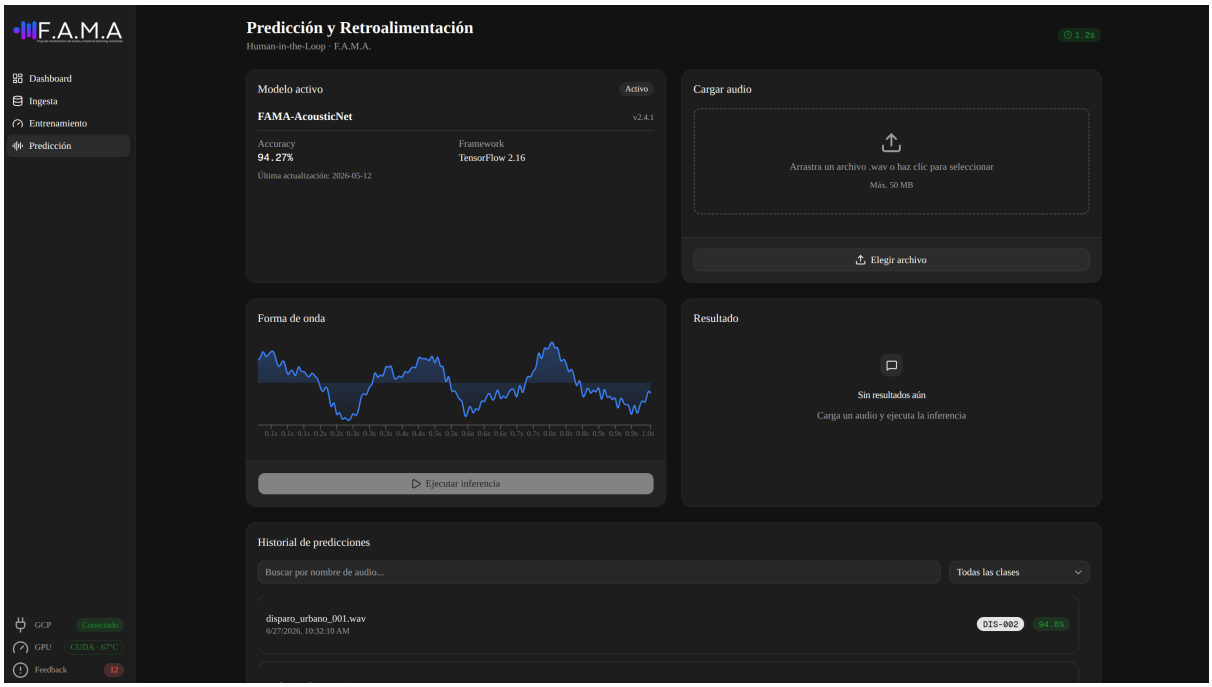


Figura 6.7: Vista de predicción de datos

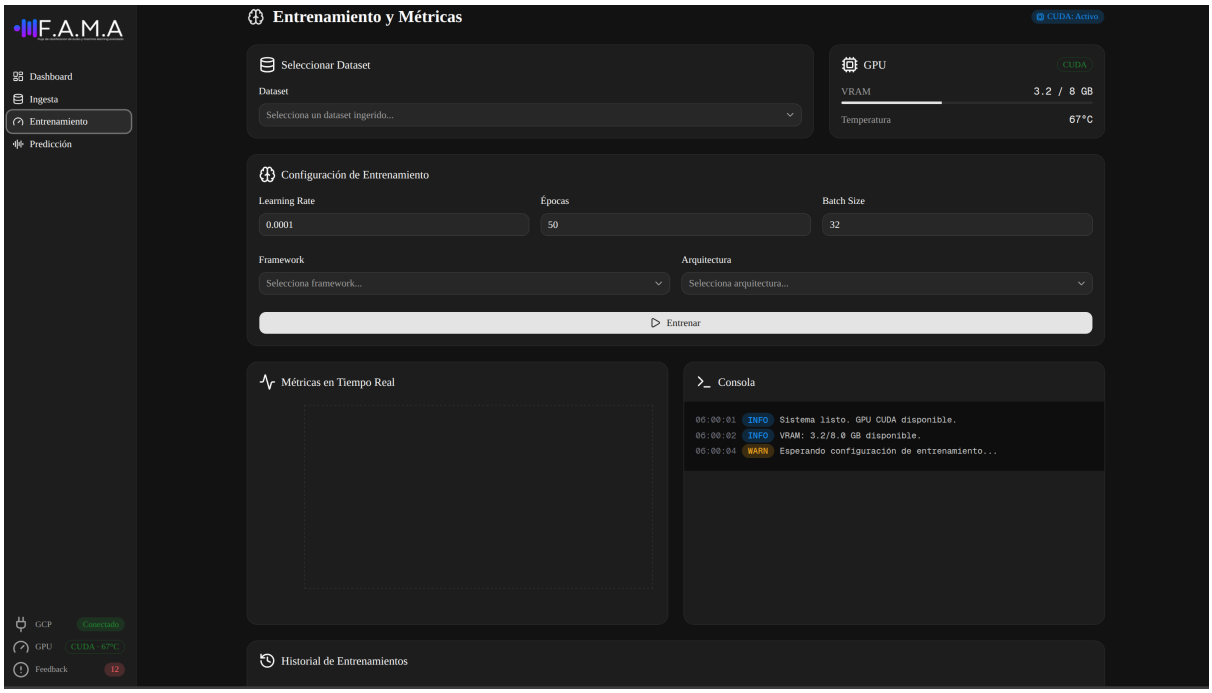


Figura 6.8: Vista de entrenamiento del modelo

6.4. Diseño de arquitectura

6.4.1. Configuración Inicial del Entorno

La infraestructura de F.A.M.A. adopta una arquitectura híbrida (mezcla de servidores propios y servicios de cloud), esto debido a la naturaleza mixta de sus cargas de trabajo: almacenamiento masivo de bajo costo y de baja frecuencia de lectura/escritura, versus cómputo tensorial intensivo con dependencia de hardware gráfico dedicado. La evaluación de alternativas se resume a continuacion

- **Servidores propios:** Descartado como única opción. Aunque entrega control total sobre el cómputo, implica costos prohibitivos de almacenamiento escalable. Además, al no contar con respaldos, requiere hardware adicional exclusivo para este propósito.
- **Hosting / Cloud-Native exclusivo:** Descartado por su inviabilidad económica en el entorno académico. Arrendar instancias con GPU NVIDIA/CUDA como nodo de cómputo incrementa significativamente los costos operativos.
- **Cloud + On-Premise (Arquitectura Híbrida):** Decisión adoptada. Se aprovechan los servicios administrados de Google Cloud Platform para la persistencia (Cloud Storage y Cloud SQL), mientras que el hardware local con GPU NVIDIA/CUDA se utiliza como nodo de cómputo, reduciendo drásticamente los costos operativos.

Capa	**Decisión**	**Servicio / Recurso**	**Justificación**
Almacenamiento de objetos	Cloud	Google Cloud Storage	Repositorio centralizado de datasets y binarios. Costos marginales y replicación.
Base de datos relacional	Cloud	Google Cloud SQL (PostgreSQL)	Integridad referencial ACID y backups automáticos.
Cómputo de entrenamiento	Servidor propio	Estación local con GPU	Acelera tensores sin pagar por hora de GPU remota.
Inferencia (predicción)	Servidor propio	API REST local (FastAPI)	Baja latencia sin viajes innecesarios a la nube.
Interfaz de usuario	Servidor propio	Next.js	SPA servida localmente durante la operatoria.

Tabla 6.15: Decisión de infraestructura por capa del sistema.

6.4.2. Arquitectura Implementación Final

El sistema se organiza en dos proyectos con responsabilidades separadas, comunicados exclusivamente por una API REST documentada.

Para su comprensión, la arquitectura se divide en dos vistas principales: física y lógica.

- **Vista física:** Muestra la topología de red distribuida en tres zonas. La estación de trabajo local aloja la interfaz gráfica (navegador) y el orquestador (FastAPI), siendo este el único con acceso a la GPU local mediante PCIe. Por otro lado, Google Cloud aloja los servicios administrados de bases de datos y almacenamiento, a los cuales el orquestador accede vía HTTPS utilizando credenciales seguras (IAM).

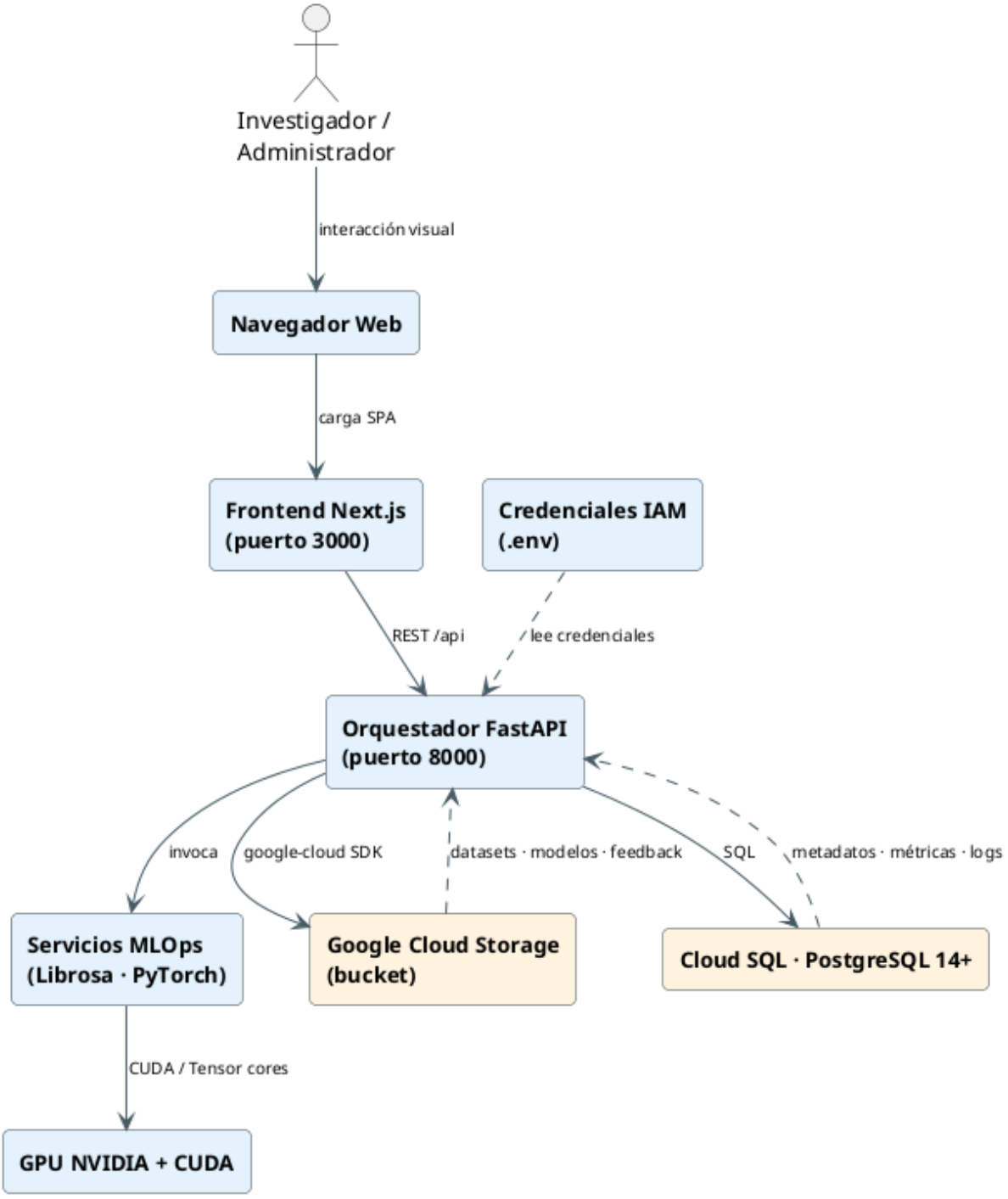


Figura 6.9: Diagrama físico de F.A.M.A

- **Vista lógica:** Abstrae los dispositivos y adopta un patrón de tres capas (Presentación, Aplicación y Datos). La capa de presentación (Next.js) se comunica con el servidot mediante solicitudes HTTP (REST). La capa de aplicación (orquestador en Python) es el corazón del framework e implementa la lógica de negocio MLOps. Finalmente, la capa de datos almacena la información estructurada en PostgreSQL y los archivos mediante Cloud Storage.

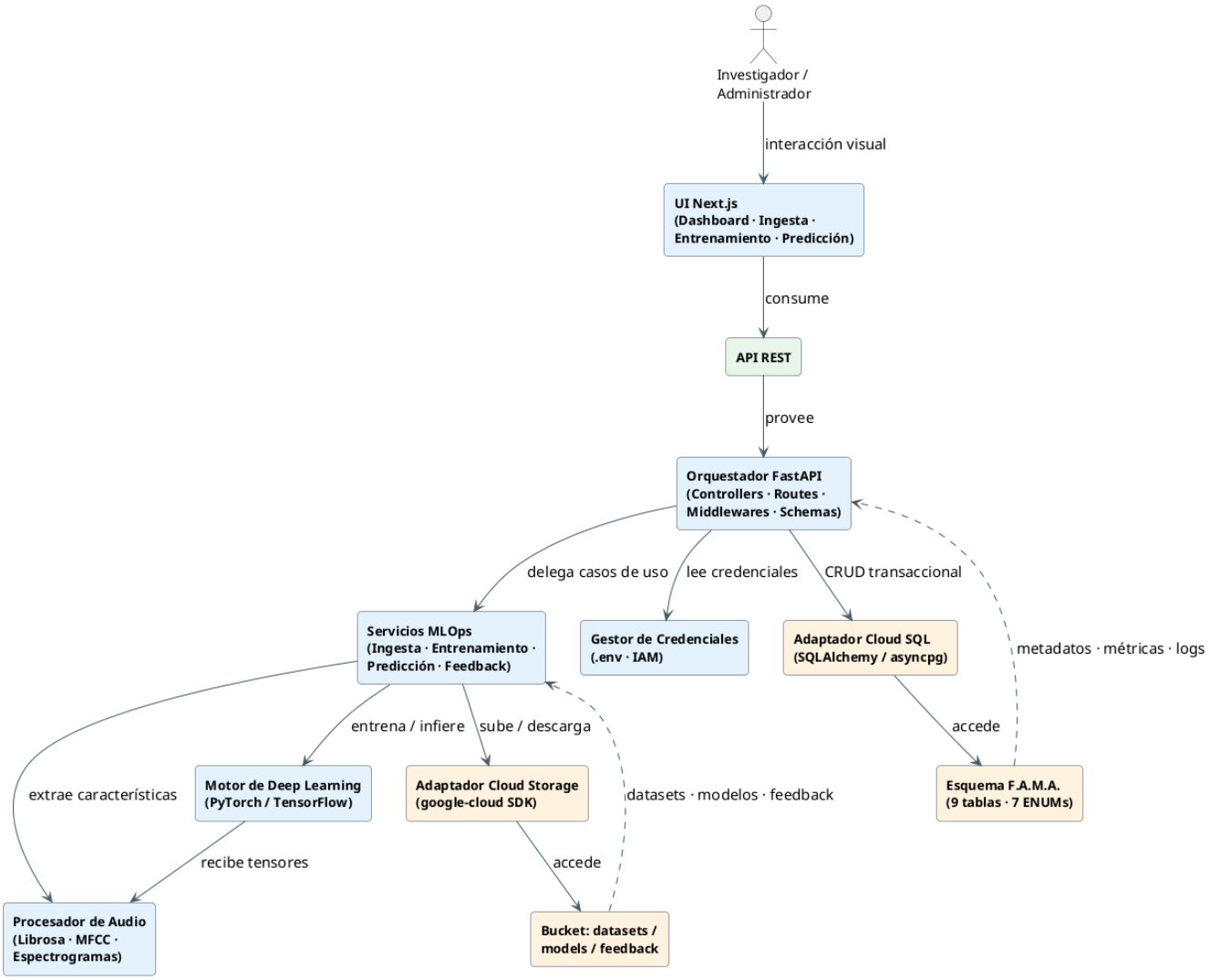


Figura 6.10: Diagrama lógico de F.A.M.A

Patrón de arquitectura

- **Backend (Orquestador:** Utiliza una Arquitectura por capas (Layered) con variantes de MVC extendido. La separación en Rutas, Controladores, Servicios y Modelos aísla las dependencias externas (GCS, Librosa, CUDA) en la capa de Servicios, manteniendo los Controladores limpios. Se utilizan Middlewares para resolver aspectos transversales como autenticación y manejo de errores.
- **Frontend (Dashboard):** Adopta un patrón de componentes sobre Single Page Application (SPA). La interfaz aísla las vistas autenticadas de los componentes reutilizables (botones, tablas, gráficos), interactuando con el backend únicamente mediante peticiones HTTP asíncronas con formato JSON.

Directorio/Módulo	**Detalle**
Controllers	Funciones que orquestan la comunicación entre las peticiones del usuario y los servicios internos.
Models	Definición de las entidades y mapeo relacional (ORM) para la persistencia de datos.
Routes	Declaración de los endpoints (API REST) y asociación con los controladores.
Middlewares	Funciones interceptoras de validación, autenticación y manejo de errores.
Services	Lógica de negocio pesada e integración con APIs externas (GCP) y hardware (GPU).

Tabla 6.16: Estructura modular del Backend.

El presente capítulo expone el estado actual de avance respecto a la planificación inicial del proyecto y establece la programación de las actividades futuras que se desarrollarán durante el próximo semestre, correspondientes a la fase de Proyecto de Título.

6.5. Porcentaje de avance por actividad

A continuación, en la Tabla 6.17, se detalla el porcentaje de progreso de cada una de las actividades definidas en la planificación inicial del semestre. Se evidencia la culminación exitosa de las etapas de levantamiento, análisis y diseño arquitectónico del framework F.A.M.A., dando paso formal a la futura fase de construcción.

Actividad (Planificación Inicial)	**% de Avance**
1. Levantamiento de requerimientos funcionales y no funcionales	100 %
2. Análisis de factibilidad (técnica, operativa, económica)	100 %
3. Especificación de Casos de Uso y Matriz de Trazabilidad	100 %
4. Diseño de Arquitectura de Software y Servicios Web	100 %
5. Diseño del Modelo Relacional y Entidad-Relación	100 %
6. Diseño de Interfaz de Usuario (Guías de estilo y Mockups)	100 %
7. Desarrollo del prototipo del Backend (Orquestador FastAPI)	0 %
8. Integración bidireccional con Google Cloud Storage y PostgreSQL	0 %
9. Desarrollo del Frontend interactivo (Dashboard)	0 %
10. Integración de pipelines de entrenamiento local e IA	0 %
11. Pruebas de integración, telemetría y ciclo de retroalimentación	0 %
12. Despliegue, manuales de usuario y documentación final	0 %

Tabla 6.17: Porcentaje de avance por actividad según la planificación inicial.

6.6. Programación de actividades futuras

Para la continuidad del proyecto durante el próximo semestre, enfocado en el Proyecto de Título, se ha actualizado la programación de tareas operativas. Estas actividades abandonan la fase de análisis para centrarse puramente en la implementación física del sistema, abarcando la programación, las pruebas de los prototipos y la validación del sistema con los investigadores.

Las principales fases a desarrollar y programar en la próxima Carta Gantt serán:

- **Fase de Construcción (Cloud y Orquestación):** Configuración de recursos en Google Cloud Platform, instanciación de la base de datos y desarrollo de las APIs del orquestador local.
- **Fase de IA y Procesamiento:** Programación de rutinas de extracción de espectrogramas y habilitación del entrenamiento de redes neuronales utilizando aceleración por GPU.
- **Fase de Interfaz Visual:** Desarrollo del *Dashboard* para la ingesta de audios y visualización de predicciones en tiempo real.
- **Fase de Pruebas y Ajustes:** Validación del mecanismo de telemetría, asegurando que las retroalimentaciones del usuario corrijan efectivamente la base de datos y almacenen los registros en la nube.

Nota: La Carta Gantt detallada con las fechas específicas de estas actividades futuras se presentará al inicio de la etapa de Proyecto de Título.



Christian Lautaro Vidal Castro
para mí

Hola Kevin
me parece bien el informe, una observación:
El modelo relacional no debe llevar cardinalidades, solo muestra las claves primarias y claves foráneas

saludos

•••

--
Christian Vidal Castro
Académico
Depto. Sistemas de Información
Universidad del Bío-Bío
Facultad de Ciencias Empresariales
Universidad del Bío-Bío, Concepción, Chile
cvidal@ubiobio.cl
+56 41 3111811

Capítulo 7

Conclusiones

- Concluir respecto a el logro de cada uno de los objetivos específicos del proyecto, y por ende se concluye respecto al logro del objetivo general "aún aquellos proyectos que según el capítulo 6 les falta implantación"
- Concluir respecto al tiempo y esfuerzo estimado y real
- Concluir respecto a logro de competencias, desarrollo de competencias del perfil de su carrera (buscar en sitio de la carrera)
- Concluir respecto a percepciones u opiniones personales obtenidas del proyecto

7.1. Trabajo Futuro

Referencias

Apéndice A

Definiciones y abreviaciones del Negocio

Detalle las palabras que se usarán más que diccionario TIC incluya una descripción de los conceptos del negocio.

Apéndice B

Pruebas de Aceptación

Este anexo SE ENTREGA EN LA REVISIÓN DEL SW. Se prueban todos los requisitos. Por cada requisito indique los pre-requisitos o configuraciones necesarios para la prueba y luego la tabla con los datos de prueba. Por ejemplo, Requerimiento REGISTRAR NUEVO PERSONAL. El usuario se logea con la cuenta usuario ENCARGADO DE PERSONAL, rut 11111111-1 contraseña clave 123, y se prueban los casos indicados, en la Error: Reference source not found.

Apéndice C

Recopilación de Información

<Todas las técnicas aplicadas y el respaldo de la información recopilada. Por ejemplo, entrevista, cuestionarios, observación en terreno, revisión de documentación, talleres grupales, etc. a. En cuestionarios se indica: a quien, para que, cuando se aplicó, y las preguntas. Después se incluyen las tablas de respuestas y resumen de los resultados. b. En entrevistas se indica: a quien, para que, cuando se aplicó, y preguntas. Después se incluyen las respuestas y firma del cliente. c. Observación en terreno: donde se realizó, que proceso observó, que usuarios, que información recopiló. d. Revisión de documentos (internos o externos): que documentos obtuvo, desde donde, cuando los obtuvo, que información útil extrajo. Toda la información que se recopila de las técnicas debe estar relacionada con el contenido del informe, es decir con la etapa del desarrollo del software. Por ejemplo, si no estamos interesados de los tipos de usuarios del software, no debe preguntar o extraer información al respecto.>

Apéndice D

Diccionario de datos

Entidades/ colecciones: que representan Atributos: que información contienen , formato, valores por defecto, reglas y validaciones

Apéndice E

Aspectos de gestión de proyectos

E.1. Carta Gantt con línea base y desviaciones

Incluir cada Gantt con la explicación del cambio y el efecto en la planificación global

E.2. Riesgos de Alto nivel (Amenazas), Impacto, estrategia

Explique los Riesgos que tengan impacto en su proyecto. Ordene los riesgos y defina las acciones (estrategia)que se proponen para abordarles. Incluir columna con los riesgos que se presentaron en el proyecto.

E.3. Estimación CU

Estimación de tamaño de Sw: Puntos de Casos de Uso • Clasificar Actores • Clasificar casos de uso • Factores técnicos • Factores del entorno • Calcular puntos de Casos de uso

Tipo de caso de uso 5 Simple Menos de 5 clases 5 3 transacciones o menos 10 Medio 5 a 10 clases 10 4 a 7 transacciones 15 Complejo Más de 10 clases 18 Más de 7 transacciones

Tipo de actor D

1 Simple Otro sistema que interactúa con el sistema a desarrollar mediante una interfaz de programación (API). 2 Medio Otro sistema interactuando a través de un protocolo (ej. TCP/IP) o una persona interactuando a través de una interfaz en modo texto 3 Complejo Una persona que interactúa con el sistema mediante una interfaz gráfica (GUI).

- Calcular UUCP (Unadjusted Use Case Point)
- UUCP= UAW+UUCW
- Calcular TCF (Technical Complexity Factor)
- TCF=0.6+(0.01*TFactor)
- Calcular EF (Environmental Factor)
- EF=1.4+(-0.03*EFactor)
- UCP = UUCP * TCF * EF

Evaluación de relevancia de factores técnicos y ambientales Valor Irrelevante De 0 a 2. Medio De 3 a 4. Esencial 5

Calculate TCF (Technical Complexity Factor)

Technical Factor Multiplier Relevancia percibida Resultado multiplicación Distributed System 2 Application performance objectives, in either response or throughput 1 End-user efficiency (on-line) 1 Complex internal processing 1 Reusability, the code must be able to reuse in other applications 1 Installation ease 0,5 Operational ease, usability 0,5 Portability 2 Changeability 1 Concurrency 1 Special security features 1 Provide direct access for third parties 1 Special user training facilities 1

Environmental Factor Multiplier Relevancia percibida Resultado multiplicación Familiar with Objectory + RUP 1,5 Application experience 0,5 Object Oriented experience 1 Analyst capability 0,5 Motivation 1 Stable requirements 2 Par time workers -1 Difficult programming language -1

Level of Effort. Schneider and Winters, proponen que: Si la suma entre (el número de factores de entorno (F1 a F6) inferiores a 3 y el número de factores de entorno (F7 a F8) superiores a 3).

- es menor o igual a 2 entonces LOE=20,
- es 3 o 4 LOE=28.
- es mayor a 4 reconsiderar el proyecto. Por ejemplo, reducir los riesgos relacionados con los factores de entorno.

E.4. Resumen Esfuerzo

El final de este documento se debe indicar las horas destinadas en realizar cada una de las fases del desarrollo del software, las horas corresponden a la suma de las horas gastadas por cada integrante y del equipo en conjunto.

Tabla

Actividades/fases/casos de Uso N° Horas Cuantas horas se dedicaron en) Cuantas horas se dedicaron en Cuantas horas se dedicaron en Cuantas horas se dedicaron en programar Cuantas horas se dedicaron en informe completo (preparar y corregir) Cuantas horas se dedicaron a git TOTAL

E.5. Retrospectiva Proyecto

- Síntesis del porcentaje de cumplimiento de los requerimientos por cada módulo.
- Análisis éxito/fracaso del proyecto
- Riesgos que se concretaron en el proyecto y efectos/consecuencias
- Análisis de ajuste entre planificación, esfuerzo real gastado y estimación de CU
- Compare y analice los resultados extraídos desde los tiempos de la Gantt, de esfuerzo requerido y estimación de CU.
- Concluya respecto a los resultados.

E.6. Iteraciones en el desarrollo

Por cada incremento y/o iteración:
- Funcionalidad
- Fecha
- Retroalimentación del cliente/usuario